

NOVEMBER 20, 2024



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

METODOLOGÍA PARA LA ELECCIÓN DE MODELOS
DE LENGUAJE DE GRAN TAMAÑO (LLMS) EN
APLICACIONES DE PREDICCIÓN
METEOROLÓGICAS A TRAVÉS DE ALGORITMOS
DE APRENDIZAJE AUTOMÁTICO SOBRE
ENTORNOS DE COMPUTACIÓN EN LA NUBE Y
CAPTURAS DE DATOS A TRAVÉS DE IOT

AGENTES IA EN IOT



INVESTIGACIÓN X

PROFUNDIZACIÓN

AUTOR

IVÁN DARIO BELLO GONZÁLEZ

Ingeniero de Telecomunicaciones

Tecnólogo en Electrónica

Universidad Distrital Francisco José de Caldas

DIRECTOR

ELVIS EDUARDO GAONA GARCÍA

Doctor en Ingeniería

Magister Ciencias de la información y las Comunicaciones

Especialista en Telecomunicaciones Móviles

Ingeniero Electrónico

Universidad Distrital Francisco José de Caldas

CODIRECTOR

PAULO ALONSO GAONA GARCÍA

Doctor en Ingeniería de la Información y del Conocimiento

Magister Ciencias de la información y las Comunicaciones

Ingeniero de Sistemas

Universidad Distrital Francisco José de Caldas

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Maestría en Ciencias de la Información y las Comunicaciones

Énfasis en Teleinformática

Grupos de Investigación GITUD y GIIRA

Bogotá, Colombia

Octubre 2024

CONTENIDO

1. LISTA DE FIGURAS	5
2. LISTA DE TABLAS	6
3. LISTA ECUACIONES	6
4. RESUMEN.....	8
5. PALABRAS CLAVE	9
6. INTRODUCCIÓN	10
7. OBJETIVO.....	12
7.1. OBJETIVO GENERAL	12
7.2. OBJETIVOS ESPECÍFICOS	12
8. MARCO DE REFERENCIA	13
8.1. MARCO TEÓRICO	13
8.2. MARCO CONCEPTUAL.....	15
9. HIPÓTESIS	18
10. AGENTES UTILIZADOS EN SISTEMAS IOT PARA MONITOREO CLIMÁTICO...19	
10.1. LANGGRAPH Y EL PARADIGMA REACT	19
10.2. IMPLEMENTACIÓN DEL AGENTE REACT EN EL SISTEMA IOT	20
10.3. HERRAMIENTAS PROPORCIONADAS AL AGENTE REACT PARA LA PREDICCIÓN METEOROLÓGICA	22
11. MÉTRICAS PARA LA EVALUACIÓN DE LA PRECISIÓN Y EXACTITUD EN PREDICCIONES METEOROLÓGICAS	32
11.1. MÉTRICAS CLÁSICAS PARA LA EVALUACIÓN DE MODELOS PREDICTIVOS	32
11.2. MÉTRICA PARA MODELOS DE LENGUAJE DE GRAN TAMAÑO (LLMS).....	34
12. SELECCIÓN DE LOS LLMS A EVALUAR EN SISTEMAS DE MONITOREO METEOROLÓGICO IOT	40

12.1.	CLAUDE-3-5-SONNET-20240620 (ANTHROPIC)	40
12.2.	GPT-4O (OPENAI).....	40
12.3.	LLAMA-V3-70B-INSTRUCT (META AI)	41
12.4.	YI-LARGE (01.AI)	41
13.	IMPORTANCIA DE LA OBSERVABILIDAD EN LLMS	42
13.1.	BENEFICIOS DEL USO DE LANGSMITH.....	44
14.	PREDICCIÓN DE VARIABLES METEOROLÓGICAS MEDIANTE RANDOM FOREST REGRESSION	46
14.1.	MODELOS DE REGRESIÓN EN PREDICCIÓN METEOROLÓGICA	46
14.2.	RANDOM FOREST REGRESSION: FUNDAMENTOS Y VENTAJAS	46
15.	METODOLOGÍA.....	49
15.1.	IDENTIFICACIÓN	49
15.2.	DESARROLLO	50
15.3.	PRUEBA Y MONITOREO	50
15.4.	EVALUACIÓN	50
15.5.	ANÁLISIS	51
16.	CASO DE ESTUDIO: MEDICIÓN DE VARIABLES CLIMATICAS EN HOME OFFICE.....	52
16.1.	IDENTIFICACIÓN	52
16.2.	DESARROLLO	60
16.3.	PRUEBAS Y MONITOREO	61
16.4.	EVALUACIÓN	64
16.5.	ANÁLISIS	71
17.	ANÁLISIS DE RESULTADOS DE LA APLICACIÓN DE LA METODOLOGÍA AL CASO DE ESTUDIO	80
10.	CONCLUSIONES.....	83

10.0. EL AGENTE IOT SU RAZONAMIENTO E INFERENCIA.....84

11. REFERENCIAS85

12. APÉNDICE89

12.0. CONFIGURACIÓN DEL GATEWAY RAK EN AWS.....89

1. LISTA DE FIGURAS

Figura 1 Grafo del Agente ReAct, creado con LangGraph en Python.....	20
Figura 2 Capacidades del Agentes ReAct, Fuente: Autores.....	21
Figura 3 Herramienta de consulta de datos Meteorológicos, Fuente: Autores.....	24
Figura 4 Herramienta de consulta de variables de comunicación, Fuente: Autores	26
Figura 5 Herramienta de obtención de fecha. Fuente: Autores	27
Figura 6 Herramienta de carga del modelo entrenado desde s3. Fuente: Autores.....	28
Figura 7 Herramienta creada para evaluar el performance del modelo MSE, RMSE, R ² , Fuente Autores	29
Figura 8 Herramienta detección de anomalías usando Isolation Forest, Fuente: Autores.	30
Figura 9 Herramienta para la consulta de variables meteorológicos usando API OpenWeather. Fuente: Autores	30
Figura 10 Cantidad de tokens utilizados por la herramienta get_weather usando tokenizer de OpenAI. Fuente: Autores.....	36
Figura 11 Interfaz de LangSmith, se muestra diferentes herramientas usadas por el agente ReAct. Fuente: Autores	44
Figura 12 Metodología para la selección de LLMs. Fuente: Autores.....	49
Figura 13 Gateway RAK7268, modulo RAK19007 / RAK4631. Fuente: Autores.....	53
Figura 14 dashboard que muestra las variables de comunicación del sensor. Hecho con Grafana. Fuente Autores.....	55
Figura 15 Dashboard creado para la visualización de los datos capturados por el sensor IoT, hecho con Grafana. Fuente Autores.....	55
Figura 16 Arquitectura serverless para aplicación de metodología para la selección de LLMs en aplicaciones de IoT. Fuente: Autores	56
Figura 17 Etapas de la plataforma para aplicación de metodología. Fuente Autores.....	58
Figura 18 flujo e interacción de los componentes. Fuente Autores	59
Figura 19 Interfaz de LangSmith plataforma usada para el monitoreo. Fuente: Autores...	60
Figura 20 Proceso de razonamiento y actuación del agente, pregunta Q1. Fuente Autores	70
Figura 21 Gráfica de radar para la pregunta Q1. Fuente: Autores	74
Figura 22 Gráfica de radar para la pregunta Q2. Fuente: Autores	75

Figura 23 Gráfica de radar para la pregunta Q3. Fuente: Autores 76

Figura 24 Gráfica de radar para la pregunta Q4. Fuente: Autores 77

Figura 25 Gráfica de radar para la pregunta Q5. Fuente: Autores 78

Figura 26 Gráfica de radar resultado de Evaluación para todos los LLMs. Fuente: Autores 79

Figura 27 Pantalla de Inicio Gateway RAK. Fuente: Autores..... 89

Figura 28 Gateway RAK, interfaz de configuración "basic station". Fuente: Autores..... 91

Figura 29 Dashboard de Gateway Rak. Fuente: Autores 93

2. LISTA DE TABLAS

Tabla 1 Muestra el total de tokens por cada una de las herramientas creadas para el Agente: Fuente: Autores 36

Tabla 2 Relación de preguntas realizadas al agente y nivel de complejidad 62

Tabla 3 Respuestas Golden Test, mejores respuestas dadas por los LLM, fuente: Autores 67

Tabla 4 Métricas cuantitativas y cualitativas de los diferentes LLMs, Pregunta Q2. Fuente: Autores..... 71

Tabla 5 Comparación de LLMs de acuerdo con las métricas seleccionadas. Fuente: Autores..... 82

Tabla 6 Probabilidad de uso de cada una de las herramientas seleccionadas por el Agente. Fuente: Autores 83

3. LISTA ECUACIONES

Ecuación 1 Error cuadrático Medio (MSE)..... 32

Ecuación 2 Raíz del Error Cuadrático Medio..... 33

Ecuación 3 Coeficiente de determinación R^2 33

Ecuación 4 Suma total de los tokens usados por docstring de las herramientas.....	34
Ecuación 5 Sumatoria de token considerando la restricción de token por modelo y contexto	35
Ecuación 6 Probabilidad de uso de la herramienta por parte del LLM	35

4. RESUMEN

La predicción meteorológica es uno de los factores críticos que se aborda desde diversos enfoques, y resulta fundamental para una amplia gama de sectores, como la agricultura, la energía renovable, la gestión de desastres y la planificación urbana. Los avances recientes en Modelos de Lenguaje de Gran Tamaño (Large Language Models, LLMs), Internet de las Cosas (Internet of Things, IoT) y computación en la nube han abierto nuevas oportunidades para mejorar la precisión y la eficiencia de las predicciones en estos sectores. Sin embargo, existen varios desafíos relacionados con la variabilidad constante de las condiciones ambientales y la fiabilidad de los datos obtenidos de sensores.

Esta investigación propone el desarrollo de una metodología integral para evaluar el impacto de la integración de LLMs con infraestructuras IoT y la computación en la nube, con el objetivo de determinar la precisión y mejorar la exactitud de las predicciones meteorológicas. La metodología comprende cinco fases iterativas: Identificación, Desarrollo, Prueba y Monitoreo, Evaluación y Análisis. Este enfoque permite la evaluación continua de los LLMs y la adaptación del sistema en función de los resultados obtenidos, abordando las necesidades cambiantes del entorno IoT.

El estudio se enfoca en el diseño de métricas específicas para evaluar el rendimiento de los LLMs frente a los modelos tradicionales, desplegado dentro de una plataforma en la nube escalable que facilita la integración de los datos generados por dispositivos IoT. La metodología incorpora el uso de un agente ReAct (Reasoning and Acting), que mejora la precisión y exactitud del sistema al detectar anomalías en los datos y ajustar las respuestas en consecuencia. Este agente también demostró ser capaz de identificar cuando el rendimiento del modelo era insuficiente, recomendando el uso de fuentes de datos más fiables como alternativa para garantizar la calidad de las predicciones.

En el caso de estudio, se evidenció que algunos modelos presentaron un bajo rendimiento, con métricas como el R^2 próximo a cero, indicando una incapacidad para capturar patrones subyacentes en los datos. Sin embargo, la inclusión del agente ReAct permitió mitigar estos problemas al tomar decisiones críticas para mantener la calidad de las predicciones. Los

resultados mostraron la capacidad del sistema para ajustarse y mejorar conforme se recopilan nuevos datos, haciendo que el proceso sea adaptativo y más robusto.

Se espera que los resultados de esta investigación contribuyan significativamente al avance de la predicción meteorológica, con beneficios directos para sectores críticos y diversas partes interesadas. La metodología desarrollada sienta las bases para futuras investigaciones y aplicaciones en este campo, facilitando predicciones meteorológicas más precisas y confiables. La combinación de LLMs con IoT y agentes reactivos no solo mejora la capacidad predictiva, sino también la adaptabilidad del sistema en entornos cambiantes, lo cual es esencial para aplicaciones meteorológicas modernas.

5. PALABRAS CLAVE

Agentes IA, ReAct, Predicción meteorológica, Modelos de Lenguaje de Gran Tamaño (LLMs), Internet de las Cosas (IoT), Computación en la nube, Aprendizaje automático (ML).

6. INTRODUCCIÓN

La oportuna predicción del clima es crucial para una diversa gama de sectores y aplicaciones abarcando desde la gestión de recursos hídricos, la agricultura, hasta las energías renovables. Sin embargo, la naturaleza compleja y dinámica de los sistemas meteorológicos presenta desafíos significativos para los enfoques tradicionales de predicción. En los últimos años, la llegada de Modelos de lenguaje de gran tamaño (MMLs), junto con los avances en IoT y la computación en la nube, han dado apertura de nuevas posibilidades para mejorar la precisión y eficiencia de las predicciones meteorológicas (J. Zhang et al., 2021) (Sri Preethaa et al., 2023).

Los LLMs, como GPT, Claude, Llama, entre otros, han demostrado capacidades para generar texto similar al humano y realizar tareas de comprensión del lenguaje natural, al tener una arquitectura subyacente basada en Transformers, es decir, una red neuronal que comprende texto se ha demostrado que tienen un gran potencial en tareas de predicción de series temporales; similar a las que se encuentran en estaciones meteorológicas, como la temperatura en días sucesivos (Brown et al., 2020) (Zhou et al., 2021) (Nascimento et al., 2023a).

Por otra parte, sistemas y desarrollos basados en IoT, han permitido la recopilación de datos meteorológicos en tiempo real, haciendo uso de sensores y proporcionando conjuntos de datos para el análisis y la predicción. De la misma forma, la computación en la nube ofrece la infraestructura necesaria para almacenar, procesar y analizar grandes volúmenes de datos de manera escalable y rentable. La integración de LLMs con infraestructuras IoT y en la nube presenta una oportunidad para aprovechar sus fortalezas y profundizar el campo de la predicción meteorológica (Zanella et al., 2014) (Petrovic et al., 2023a) (Dai et al., 2023a).

Sin embargo, la aplicación de LLMs en la predicción del tiempo conlleva varios desafíos. Estos incluyen la necesidad de identificar y validar métricas de evaluación apropiadas para medir la precisión y exactitud de las predicciones generadas por los LLMs en comparación con los modelos convencionales (Lee et al., 2022). Además, se requiere el desarrollo de una plataforma en la nube que permita el almacenamiento y procesamiento escalable de

los datos recopilados por dispositivos IoT, garantizando que los LLMs puedan acceder y utilizar estos datos de manera eficiente. Asimismo, es fundamental definir una metodología que abarque las diferentes fases de análisis para la elección de los LLMs, incluyendo la recolección y preprocesamiento de datos capturados a través de IoT, y la validación de la metodología mediante casos de estudio reales.

En este contexto, el presente documento de investigación propone el desarrollo de una metodología para evaluar el impacto de la integración de LLMs en aplicaciones de predicción meteorológicas, utilizando algoritmos de aprendizaje automático sobre entornos de Internet de las Cosas (IoT) y computación en la nube. El estudio tiene como objetivo abordar los desafíos clave asociados con la elección y aplicación de LLMs en este ámbito, proporcionando un marco que permita evaluar su rendimiento en relación con los modelos de aprendizaje automático convencionales. Además, se busca validar la metodología desarrollada mediante la aplicación de escenarios basados en casos de estudio reales, comparando los resultados obtenidos con los métodos tradicionales. De esta manera, se espera contribuir a la toma de decisiones informadas que favorezcan el despliegue de aplicaciones de predicción meteorológica basadas en LLMs, IoT y computación en la nube.

7. OBJETIVO

7.1. OBJETIVO GENERAL

Desarrollar una metodología que permita evaluar el impacto de la integración de Modelos de Lenguaje de Gran Tamaño (LLMs) mediante el uso de los modelos de aprendizaje automático convencionales para mejorar la precisión y exactitud de predicciones meteorológicas basadas en la recolección de datos mediante Internet de las cosas IoT e a infraestructura de computación en la nube.

7.2. OBJETIVOS ESPECÍFICOS

- Identificar, seleccionar y validar un conjunto de métricas relevantes para medir la precisión y exactitud de las predicciones meteorológicas realizadas por LLMs en comparación con los modelos convencionales.
- Desarrollar una plataforma en la nube que permita facilitar soluciones de computación y almacenamiento escalables para las cargas de datos generadas por dispositivos IoT, asegurando que los LLM puedan acceder y procesar estos datos de manera eficiente.
- Definir una metodología que permita incluir las diferentes fases de análisis para la elección de los LLM abarcando la recolección y preprocesamiento de datos capturados a través de Internet de las Cosas (IoT) y la validación de la metodología mediante casos de estudio reales.
- Validar la metodología desarrollada mediante la aplicación de escenarios basados en casos de estudio frente a resultados obtenidos mediante métodos tradicionales.

8. MARCO DE REFERENCIA

8.1. MARCO TEÓRICO

8.1.1. MODELOS DE LENGUAJE DE GRAN TAMAÑO (LLMs)

Los LLMs han surgido como un avance significativo en el campo del Procesamiento del lenguaje natural (Natural language processing, NLP). Estos modelos, como GPT, Claude, Llama, entre otros, se entrenan con muchos textos y han demostrado una capacidad para generar respuestas similares a las humanas según las indicaciones dadas. El desarrollo de LLMs se basa en la arquitectura del Transformador, introducida por Vaswani en *“Attention is All You Need”*. Esta arquitectura se aleja de las técnicas anteriores basadas en recurrencia y convoluciones, utilizando en su lugar mecanismos de atención que facilitan el modelado de dependencias sin importar la distancia en el texto. Estos mecanismos permiten a los modelos gestionar de manera más efectiva la información contextual a largo plazo, lo que es crucial para generar y comprender texto coherente (Brown et al., 2020; Vaswani et al., 2017).

Además, Los LLMs han sido aplicados en una variedad de tareas de NLP, desde la traducción automática hasta la generación de contenido y la simulación de diálogos. De la misma forma, estos modelos han demostrado ser versátiles, adaptándose a nuevas tareas mediante técnicas como el aprendizaje por transferencia y el ajuste fino con conjuntos de datos específicos del dominio. La aplicación de LLMs se ha extendido más allá de las tareas de NLP y se ha explorado en diversos dominios, incluida la predicción de series de tiempo. Los estudios han demostrado el potencial de los modelos basados en Transformers para capturar patrones temporales complejos y generar predicciones precisas. En particular, la integración de LLMs en sistemas de múltiples agentes, conocida como "GPT-in-the-loop", ha surgido como un enfoque para mejorar las capacidades de razonamiento y adaptación de estos sistemas (Brown et al., 2020; Nascimento et al., 2023b; Zhou et al., 2021).

8.1.2. INTERNET DE LAS COSAS (IOT) PARA LA RECOPIACIÓN DE DATOS METEOROLÓGICOS

El IoT se refiere a la red de dispositivos físicos, sensores y actuadores conectados que pueden recopilar, intercambiar y actuar sobre datos. En el contexto de la predicción del tiempo, los sensores IoT desplegados en estaciones meteorológicas pueden recopilar continuamente diversos parámetros, como temperatura, humedad, presión barométrica, velocidad y dirección del viento. Estos datos se pueden transmitir a través de redes de comunicación, como redes de área amplia de baja potencia (Low Power Wide Area Network, LPWAN), para su posterior procesamiento y análisis (Mekki et al., 2019; Petrovic et al., 2023b).

La integración de dispositivos IoT en la recopilación de datos meteorológicos ofrece varias ventajas. Permite la recopilación de datos en tiempo real, de alta resolución y de ubicación específica, lo que mejora la calidad y granularidad de los datos disponibles para los modelos de predicción. Además, las redes IoT pueden cubrir áreas geográficas extensas, incluidas ubicaciones remotas, lo que mejora la cobertura espacial de las observaciones meteorológicas (Tokognon et al., 2017).

8.1.3. COMPUTACIÓN EN LA NUBE PARA EL PROCESAMIENTO Y ALMACENAMIENTO DE DATOS

La computación en la nube se refiere al modelo de entrega de servicios informáticos a través de Internet, que permite el acceso a recursos compartidos, software y datos a pedido. En el contexto de la predicción meteorológica, las plataformas en la nube ofrecen una infraestructura escalable y flexible para el procesamiento, almacenamiento y análisis de grandes volúmenes de datos generados por dispositivos IoT.(Mell & Grance, 2011; PremSankar et al., 2018)

Las plataformas en la nube proporcionan varias ventajas para las aplicaciones de IoT. Ofrecen capacidades de almacenamiento virtualmente ilimitadas, lo que permite la recopilación y retención de datos históricos a largo plazo. Además, los servicios en la nube proporcionan recursos informáticos a pedido, lo que permite el procesamiento y análisis escalables de datos en tiempo real o casi real. Esto es especialmente beneficioso para la

capacitación y aplicación de modelos de aprendizaje automático, como LLMs, que requieren una potencia de cálculo significativa (Satyanarayanan, s/f; Yu et al., 2017).

La integración de IoT y la computación en la nube ha permitido el desarrollo de sistemas de predicción meteorológica mejorados. Al aprovechar los datos recopilados por los sensores IoT y el procesamiento y análisis escalables habilitados por las plataformas en la nube, estos sistemas pueden generar predicciones más precisas y oportunas. Sin embargo, esta integración también presenta retos, como la gestión eficiente de grandes volúmenes de datos, la garantía de la calidad, la confiabilidad de los datos y problemas de seguridad y privacidad (Liao et al., 2018)(Lin et al., 2017).

8.2. MARCO CONCEPTUAL

Modelo de lenguaje de gran tamaño (LLM): un modelo de lenguaje entrenado en unas grandes cantidades de texto, capaz de generar texto similar al humano y realizar tareas de comprensión del lenguaje natural.

Transformador: una arquitectura de modelo de aprendizaje profundo que utiliza mecanismos de atención para procesar y generar secuencias, comúnmente utilizada en LLMs. Esta arquitectura permite a los modelos gestionar de manera más efectiva la información contextual a largo plazo, crucial para generar y comprender texto coherente.

Internet de las cosas (IoT): una red de dispositivos físicos, sensores y actuadores interconectados que pueden recopilar e intercambiar datos.

Sensor meteorológico IoT: un dispositivo equipado con sensores que puede medir y transmitir parámetros meteorológicos, como temperatura, humedad, presión barométrica, velocidad y dirección del viento.

Computación en la nube: un modelo para entregar servicios informáticos a través de Internet, que permite el acceso a recursos, software y datos compartidos a pedido.

Plataforma en la nube: una infraestructura basada en la nube que proporciona recursos y servicios informáticos para el almacenamiento, procesamiento y análisis de datos.

Aprendizaje automático: un subcampo de la inteligencia artificial que permite a los sistemas aprender y mejorar a partir de datos sin ser programados explícitamente.

Generación Aumentada por Recuperación (RAG): un marco de trabajo para crear aplicaciones de IA generativa que pueden utilizar fuentes de datos empresariales y bases de datos vectoriales para superar limitaciones de conocimiento. RAG funciona usando un módulo recuperador para encontrar información relevante en un almacén de datos externo en respuesta a la consulta de un usuario. Estos datos recuperados se utilizan como contexto, junto con la consulta original, para crear una consulta ampliada que se pasa al modelo de lenguaje, el cual genera una respuesta que incorpora el conocimiento empresarial.

Agentes de LangChain: componentes flexibles que pueden acceder a una variedad de recursos y seleccionar cuáles usar en función de la entrada del usuario. Los agentes pueden utilizar múltiples herramientas y usar la salida de una como entrada para la siguiente. Esto permite la creación de aplicaciones adaptables que pueden realizar tareas complejas al encadenar llamadas a modelos de lenguaje y utilidades.

Patrones arquitectónicos para aplicaciones de IA generativa

- ❖ **Generación de texto:** se pasa una petición a un modelo fundacional usando un playground o una llamada API. El modelo genera texto acorde a la petición de entrada. Se puede agregar una capa de LangChain para casos de uso específicos de generación de texto.
- ❖ **Resumen de texto:** para documentos pequeños, el texto se pasa directamente al LLM, que responde con un resumen. Para documentos largos, se utiliza una arquitectura map-reduce, dividiendo el documento en fragmentos, resumiendo cada fragmento con el LLM y combinando los resúmenes iterativamente.
- ❖ **Respuesta a preguntas:** para casos de uso genéricos, el usuario envía una petición al modelo fundacional y obtiene respuestas basadas en información genérica. Para casos de uso específicos y personalizados, se utiliza RAG para recuperar fragmentos relevantes de una base de conocimiento y pasarlos al modelo fundacional junto con la pregunta del usuario para generar una respuesta precisa.
- ❖ **Chatbots:** el usuario realiza una consulta al chatbot, que pasa el historial de chat (si existe) y la consulta actual al model. El modelo genera una respuesta, que se devuelve

al usuario. Para chatbots conscientes del contexto, la consulta del usuario se convierte en un vector de incrustación usando, se realiza una búsqueda por similitud en una base de datos vectorial para recuperar fragmentos relevantes, y estos se pasan al modelo fundacional junto con la consulta y el historial de chat para generar una respuesta.

- ❖ **Generación de código:** el usuario proporciona una petición en texto sin formato que incluye una instrucción indicando al modelo qué generar y, en algunos casos, ejemplos de código. El modelo fundacional procesa la entrada y devuelve el código generado.
- ❖ **Agentes de LangChain:** los agentes tienen acceso a diversas herramientas y seleccionan cuáles usar en función de la entrada del usuario. Pueden utilizar la salida de una herramienta como entrada para la siguiente. La técnica ReAct (Reasoning and Acting) se utiliza para que el agente evalúe la petición, determine el siguiente paso para resolver el problema, ejecute ese paso y repita el proceso hasta que pueda responder la pregunta.

9. HIPÓTESIS

La integración de Modelos de Lenguaje de Gran Tamaño (LLMs) con la recolección de datos basada en IoT y la infraestructura de computación en la nube mejorará significativamente la precisión y exactitud de las predicciones meteorológicas, en comparación con las metodologías que emplean modelos de aprendizaje automático convencionales.

10. AGENTES UTILIZADOS EN SISTEMAS IOT PARA MONITOREO CLIMÁTICO

En el contexto de la evaluación y selección de LLMs para sistemas IoT, uno de los elementos clave del enfoque metodológico es el uso de un agente basado en el paradigma Razonamiento y actuación (Reasoning and Acting, ReAct), implementado a través del framework LangGraph. A continuación, se presenta una descripción detallada de estos componentes y cómo se integran para mejorar las capacidades de toma de decisiones en sistemas IoT.

10.1. LANGGRAPH Y EL PARADIGMA REACT

LangGraph es un framework desarrollado para mejorar las capacidades de los LLMs mediante la integración de herramientas avanzadas para la representación y análisis de datos lingüísticos. De acuerdo con la documentación oficial de LangGraph, el framework proporciona una infraestructura que permite a los modelos de lenguaje construir y manipular representaciones complejas de información, facilitando la integración de múltiples fuentes de datos y la utilización de diferentes herramientas de NLP. LangGraph se centra en la eficiencia y flexibilidad del procesamiento, permitiendo a los modelos de lenguaje ejecutar consultas complejas y realizar operaciones de razonamiento avanzado, lo cual es crucial en aplicaciones como la búsqueda semántica, la recuperación de información y el análisis contextual profundo (langchain, 2024).

Por otro lado, el paradigma Reasoning and Acting se enfoca en la integración de procesos de razonamiento con comportamientos orientados a la acción en los sistemas de inteligencia artificial, como se muestra en la Figura 1. ReAct enfatiza la importancia de combinar el razonamiento lógico con la toma de decisiones prácticas para crear agentes de IA más robustos y adaptables. Esta aproximación es especialmente relevante en escenarios donde los sistemas de IA deben navegar en entornos complejos, tomar decisiones basadas en información incompleta e interactuar con humanos u otros agentes.

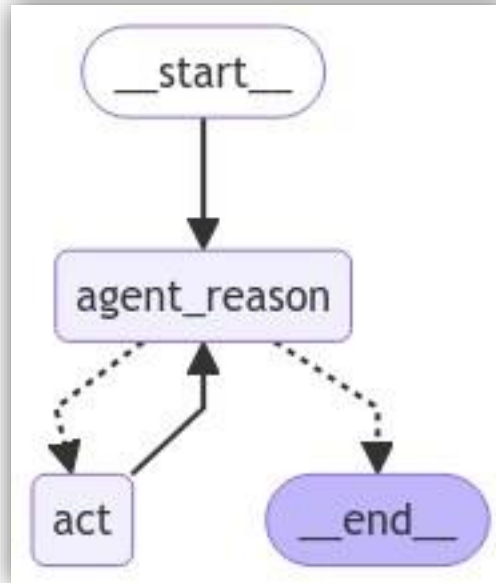


Figura 1 Grafo del Agente ReAct, creado con LangGraph en Python

De acuerdo con (Yao et al., 2022), el enfoque ReAct permite a los modelos de lenguaje no solo responder consultas, sino también razonar sobre el contexto y tomar acciones que mejoren la calidad de las respuestas. Esto se logra a través de una sinergia entre el razonamiento y la acción, donde el agente tiene la capacidad de aprender de las interacciones pasadas y ajustar sus decisiones de manera dinámica, lo cual resulta esencial en entornos cambiantes como los sistemas IoT. Este enfoque no solo mejora la precisión en la generación de respuestas, sino que también incrementa la eficiencia en el uso de recursos, factor crucial en aplicaciones IoT donde la capacidad de procesamiento y almacenamiento es limitada.

10.2. IMPLEMENTACIÓN DEL AGENTE REACT EN EL SISTEMA IOT

En el sistema de monitoreo climático IoT, el agente ReAct se implementa mediante el uso de LangGraph y está diseñado para mejorar la capacidad de los modelos de lenguaje en tareas de toma de decisiones dentro del entorno IoT. Este agente permite al sistema no solo interpretar datos en tiempo real provenientes de sensores, sino también actuar sobre

esa información de manera adecuada, considerando los recursos limitados característicos de las aplicaciones IoT.

La combinación de LangGraph y ReAct facilita que el agente pueda, como se muestra la Figura 2:

- Inferir relaciones causales entre diferentes variables climáticas (por ejemplo, temperatura, presión y humedad).
- Planificar secuencias de acción basadas en esos datos.
- Adaptarse a situaciones cambiantes en tiempo real, proporcionando respuestas robustas y precisas incluso ante información parcial o ruidos en los datos de entrada.

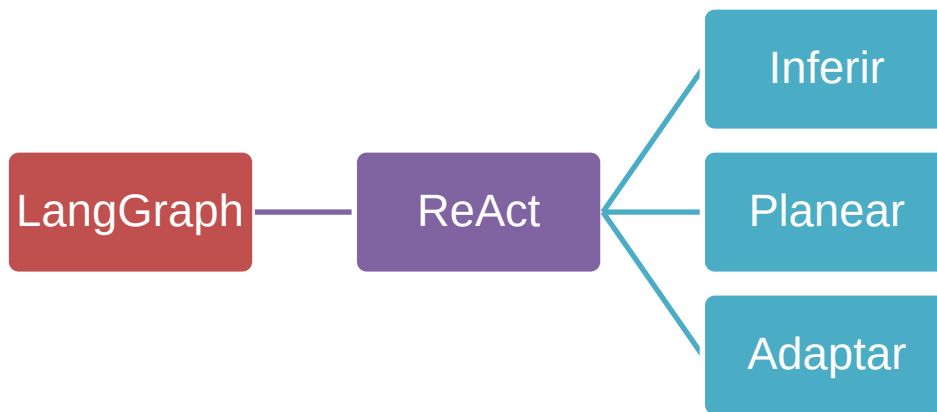


Figura 2 Capacidades del Agentes ReAct, Fuente: Autores

Este enfoque permite optimizar el uso de recursos de los modelos de lenguaje en un entorno con limitaciones de procesamiento y conectividad, lo cual es crucial para lograr eficiencia y efectividad en los despliegues IoT a gran escala.

10.3. HERRAMIENTAS PROPORCIONADAS AL AGENTE REACT PARA LA PREDICCIÓN METEOROLÓGICA

La integración de herramientas especializadas en el agente ReAct constituye un paso fundamental para optimizar la predicción meteorológica en sistemas IoT. Esta sección presenta una descripción detallada de las herramientas incorporadas en el agente ReAct, implementado con el modelo de lenguaje GPT-4o, y explica su relevancia en el contexto de sistemas de monitoreo del clima.

Estas herramientas, desarrolladas con el soporte de la biblioteca LangChain y varios servicios de AWS, permiten al agente gestionar datos meteorológicos de manera eficaz, facilitando tanto la predicción como la detección de anomalías. A continuación, se exploran cada una de estas herramientas y su papel en la mejora de la exactitud y eficiencia de los pronósticos meteorológicos.

10.3.1. HERRAMIENTA DE CONSULTA DE DATOS METEOROLÓGICOS

Una de las herramientas esenciales del agente ReAct es la capacidad de acceder y recuperar datos climáticos en tiempo real desde la base de datos Timestream de AWS. La herramienta **get_weather** se utiliza para consultar y proporcionar valores actuales de variables climáticas, Figura 3, tales como:

- **Temperatura:** Valor que indica la cantidad de calor en el ambiente, medido en grados Celsius (°C). Es una de las variables fundamentales para determinar el estado del clima y predecir posibles cambios.
- **Luz (Light):** Intensidad de luz ambiental, medida en lumens. Esta variable es importante para el análisis de la radiación solar y su impacto en la temperatura y la evapotranspiración.
- **Altitud de la base de nubes (Cloud Base Altitude):** Altitud a la que se encuentra la base de las nubes, lo cual puede influir en la previsión de eventos climáticos como lluvias o niebla.
- **Punto de rocío (Dew Point):** Temperatura a la cual el aire debe enfriarse para que el vapor de agua se condense en gotas de agua. Este valor es clave para estimar la humedad relativa y las condiciones de formación de niebla o rocío.
- **Presión de vapor actual (Actual Vapor Pressure):** Presión ejercida por el vapor de agua presente en el aire. Esta variable es relevante para analizar el contenido de humedad en la atmósfera.
- **Índice de calor (Heat Index):** Valor que representa cómo se siente la temperatura cuando se combina con la humedad. Este índice es crucial para evaluar el riesgo de estrés térmico en condiciones de calor extremo.
- **Presión de vapor de saturación (Saturation Vapor Pressure):** La presión máxima que el vapor de agua puede ejercer a una temperatura dada sin que ocurra condensación. Es un indicador importante para el análisis de la humedad del ambiente.

- **Presión (Pressure):** Presión atmosférica medida en hectopascales (hPa). La presión es un indicador del estado de la atmósfera y es útil para prever cambios en el clima, como la aproximación de sistemas de alta o baja presión.
- **Humedad (Humidity):** Cantidad de vapor de agua en el aire, expresada como un porcentaje. La humedad afecta directamente la sensación térmica y la formación de nubes y precipitaciones.
- **Humedad relativa (Relative Humidity):** Relación entre la cantidad de vapor de agua presente en el aire y la cantidad máxima que podría contener a la misma temperatura. Es un indicador clave para entender la capacidad del aire de retener humedad y predecir fenómenos como la lluvia.

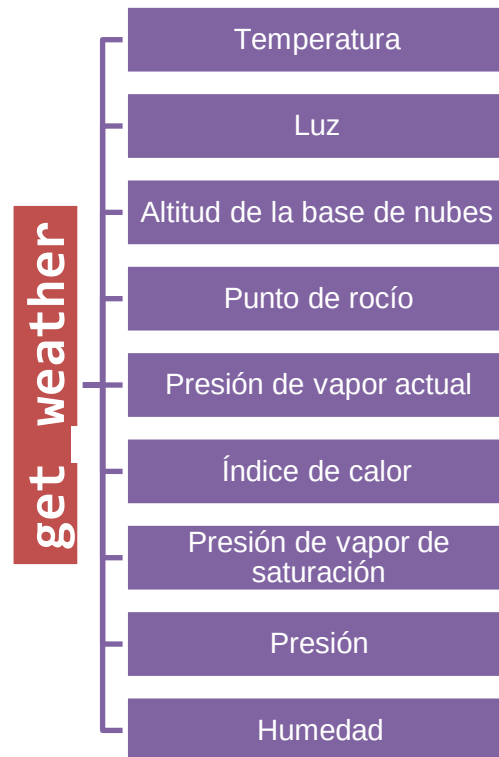


Figura 3 Herramienta de consulta de datos Meteorológicos, Fuente: Autores

Esta información se obtiene de sensores conectados que capturan datos y los almacenan en una base de datos para series temporales AWS Timestream.

La importancia de esta herramienta radica en que permite al agente analizar el estado actual del clima, proporcionando datos que son esenciales para el cálculo de predicciones. Con un enfoque en el monitoreo en tiempo real, esta herramienta facilita la adaptación dinámica de los modelos a condiciones climáticas cambiantes, garantizando que las predicciones se basen en información actualizada y confiable.

10.3.2. HERRAMIENTA DE CONSULTA DE VARIABLES DE COMUNICACIÓN

Otra herramienta creada para ser usada por el agente es **get_communication**, que se encarga de recuperar información relativa a las variables de comunicación del sistema IoT, en la Figura 4 se muestra el diagrama de la herramienta. A continuación, se describen las principales variables de comunicación que se pueden consultar:

- **RSSI (Received Signal Strength Indicator):** Indicador de la intensidad de la señal recibida, medido en dBm. Esta métrica sirve para evaluar la calidad de la conexión inalámbrica entre los sensores y el servidor, permitiendo identificar si la señal es suficientemente fuerte para garantizar la transmisión de datos sin interrupciones.
- **SNR (Signal-to-Noise Ratio):** Relación señal/ruido, medida en decibelios (dB). Un valor alto de SNR indica que la señal es mucho más fuerte que el ruido de fondo, lo cual es crucial para la integridad de los datos transmitidos. Un bajo SNR puede indicar problemas en la calidad de la comunicación, lo cual podría afectar la precisión de los datos meteorológicos.
- **Freq (Frequency):** Frecuencia de la señal utilizada por los dispositivos IoT, medida en megahercios (MHz). Conocer la frecuencia de operación es importante para asegurar que los dispositivos funcionen dentro de las bandas autorizadas y evitar interferencias con otros sistemas de comunicación.
- **fCnt (Frame Count):** Contador de tramas que mide el número de paquetes de datos transmitidos. Este valor permite monitorear la cantidad de datos enviados y detectar posibles pérdidas de paquetes o retransmisiones, lo cual es útil para garantizar una comunicación eficiente y precisa.

Estos indicadores permiten evaluar la calidad de la comunicación entre los dispositivos IoT y el servidor en la nube.

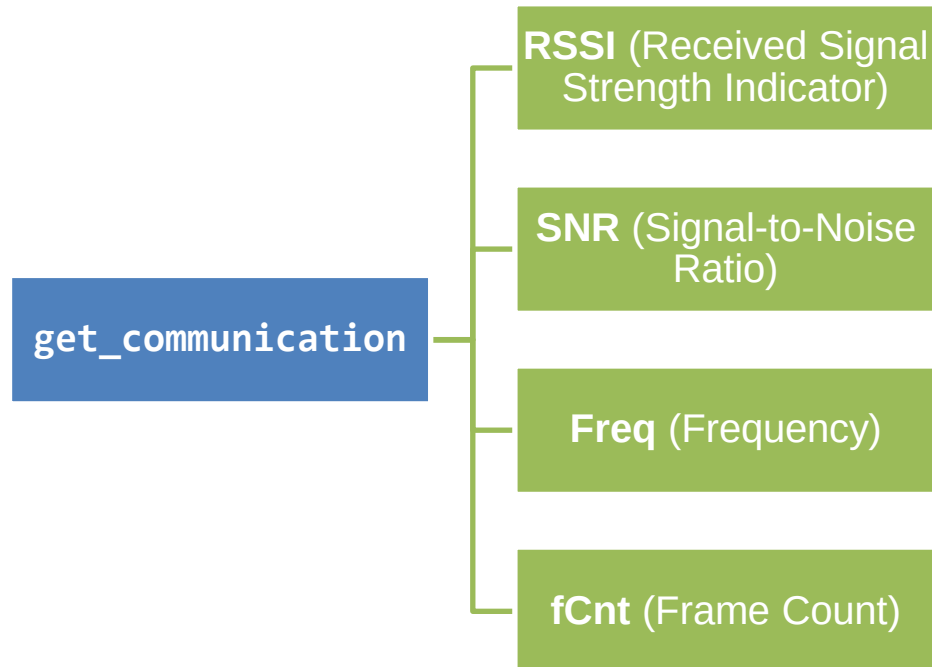


Figura 4 Herramienta de consulta de variables de comunicación, Fuente: Autores

La capacidad de monitorear y evaluar la calidad de la comunicación es crítica para el correcto funcionamiento de un sistema de monitoreo meteorológico. Problemas en la transmisión de datos podrían afectar la exactitud de las predicciones, ya que el agente depende de información precisa y constante. Por lo tanto, esta herramienta asegura que el flujo de datos entre los sensores y el agente se mantenga óptimo, minimizando el riesgo de errores debido a la pérdida de información.

10.3.3. HERRAMIENTA DE OBTENCIÓN DEL TIEMPO (FECHA)

La herramienta **get_current_time** permite al agente recuperar la hora actual en UTC en formato ISO 8601. Esta funcionalidad es útil para obtener una referencia temporal precisa, que puede ser utilizada en diferentes análisis y procesos dentro del sistema IoT, en la Figura 5 se muestra un diagrama del uso de la herramienta.

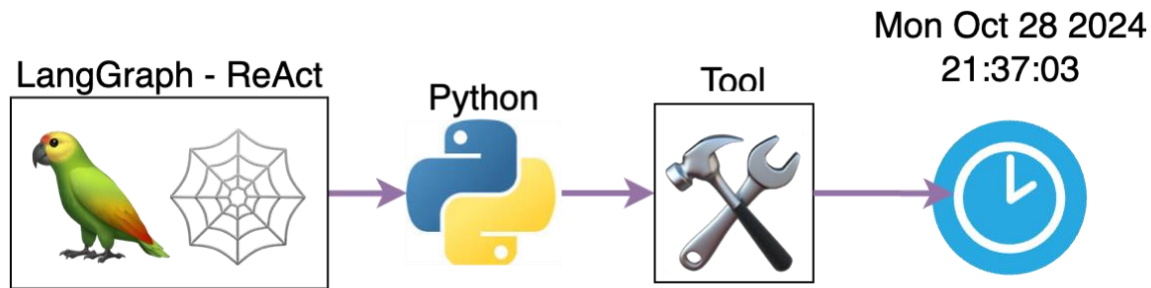


Figura 5 Herramienta de obtención de fecha. Fuente: Autores

10.3.4. HERRAMIENTA DE PREDICCIÓN DE TEMPERATURA BASADA EN MODELOS ENTRENADOS

La herramienta **load_model_and_predict** permite cargar un modelo previamente entrenado para realizar predicciones meteorológicas basadas en una serie de características, como la hora del día, la humedad, la presión, y la intensidad de luz. Utilizando modelos de aprendizaje automático alojados en Amazon S3, Figura 6, el agente puede predecir la temperatura para la siguiente hora con base en las condiciones actuales.

Esta herramienta es esencial para la generación de pronósticos meteorológicos a corto plazo, lo cual resulta crítico en aplicaciones como la agricultura inteligente, la gestión de recursos hídricos y la prevención de desastres naturales. Al contar con la capacidad de predecir la temperatura con alta precisión, el sistema IoT puede ofrecer alertas tempranas y recomendaciones basadas en datos, mejorando la capacidad de respuesta a cambios meteorológicos abruptos.

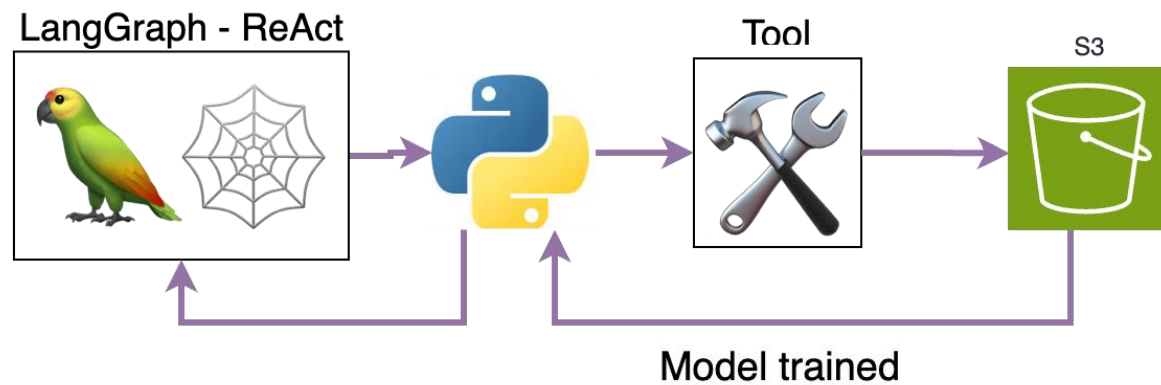


Figura 6 Herramienta de carga del modelo entrenado desde s3. Fuente: Autores

10.3.5. HERRAMIENTA DE EVALUACIÓN DEL RENDIMIENTO DEL MODELO

La herramienta `evaluate_model_performance` permite evaluar el rendimiento del modelo de predicción utilizando métricas como el Error Cuadrático Medio (MSE), la Raíz del Error Cuadrático Medio (RMSE) y el Coeficiente de Determinación (R^2), Figura 7. Estas métricas proporcionan una medida cuantitativa de la precisión del modelo y permiten identificar posibles áreas de mejora.

En el contexto de la predicción meteorológica, la evaluación continua del rendimiento del modelo es crucial para asegurar que las predicciones se mantengan confiables a lo largo del tiempo. La capacidad de detectar desviaciones o errores en las predicciones permite ajustar y mejorar el modelo de forma iterativa, lo que contribuye a una mejor adaptación a las condiciones climáticas locales.

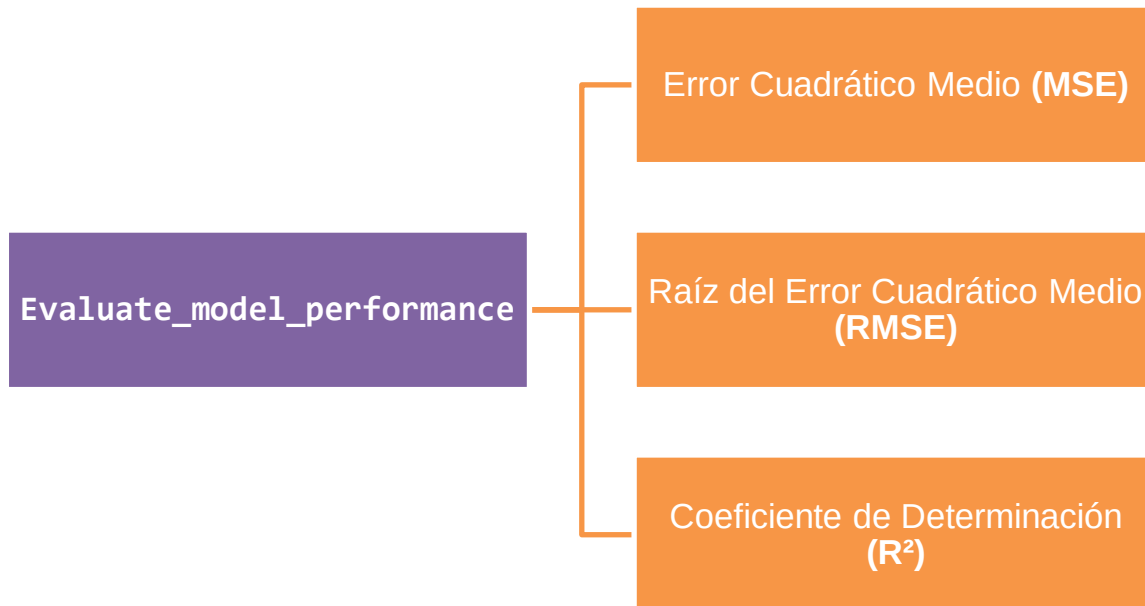


Figura 7 Herramienta creada para evaluar el performance del modelo MSE, RMSE, R^2 , Fuente Autores

10.3.6. HERRAMIENTA DE DETECCIÓN DE ANOMALÍAS

La herramienta **detect_anomalies_from_iot_sensor** utiliza el modelo de aislamiento *Isolation Forest* para detectar valores anómalos en las lecturas de los sensores meteorológicos, Figura 8. *Isolation Forest* es un algoritmo de aprendizaje no supervisado que se especializa en la detección de anomalías a través del aislamiento de puntos de datos que se consideran atípicos. Este modelo crea múltiples árboles de decisión y mide la facilidad con la cual los puntos son aislados, lo cual permite identificar aquellos valores que son potencialmente anómalos con mayor precisión. Esta herramienta analiza los datos recogidos durante las últimas 24 horas y es capaz de identificar comportamientos atípicos que podrían indicar fallas en los sensores o condiciones meteorológicas extremas.

El algoritmo *Isolation Forest* es particularmente adecuado para sistemas de monitoreo meteorológico debido a su eficiencia y capacidad para trabajar con grandes volúmenes de

datos en tiempo real, lo cual lo convierte en una herramienta robusta para mantener la integridad de los datos recopilados.(Liu et al., 2008).

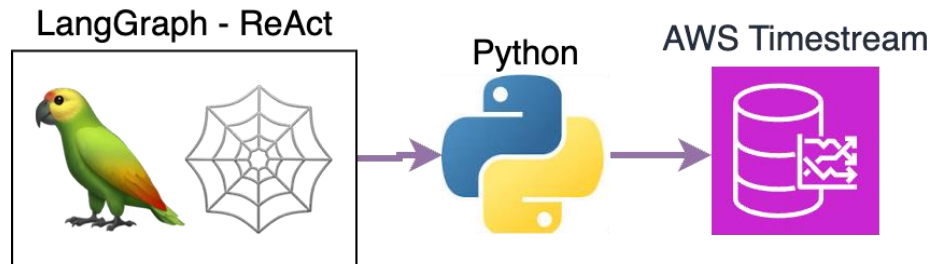


Figura 8 Herramienta detección de anomalías usando Isolation Forest, Fuente: Autores

10.3.7. HERRAMIENTA DE CONSULTA A OPENWEATHERMAP

La herramienta `get_weather_openweathermap` permite al agente acceder a información climática proporcionada por la API de OpenWeatherMap, incluyendo variables como la temperatura, la humedad, la presión atmosférica y la velocidad del viento, Figura 9. Esta fuente de datos externa complementa la información obtenida de los sensores locales, proporcionando una visión más completa de las condiciones climáticas actuales.

La integración de datos de fuentes externas permite validar y contrastar las lecturas de los sensores propios del sistema, mejorando la robustez y la fiabilidad de las predicciones. Además, ofrece una perspectiva regional que puede ser crucial para entender fenómenos climáticos de mayor escala y su impacto local.

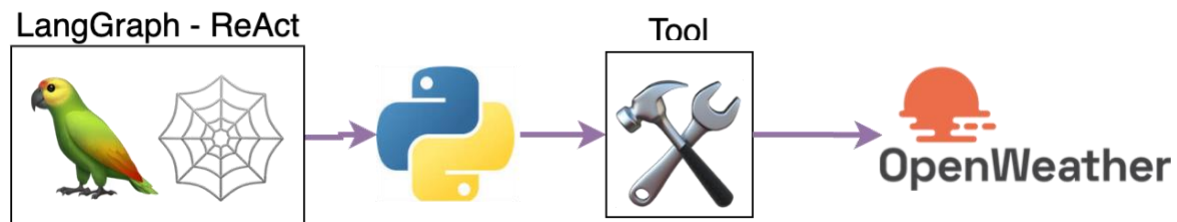


Figura 9 Herramienta para la consulta de variables meteorológicas usando API OpenWeather. Fuente: Autores

10.3.8. IMPORTANCIA DE LAS HERRAMIENTAS PARA LA PREDICCIÓN METEOROLÓGICA

La incorporación de estas herramientas al agente ReAct aporta un valor significativo en la capacidad del sistema IoT para predecir y responder a eventos climáticos. La combinación de datos en tiempo real, modelos predictivos y capacidades de detección de anomalías permite un monitoreo constante y preciso del ambiente. Esta infraestructura no solo mejora la capacidad de respuesta ante condiciones adversas, sino que también optimiza la gestión de recursos y la planificación basada en condiciones climáticas.

El uso de herramientas especializadas dentro del agente ReAct proporciona un enfoque integral para la predicción meteorológica, integrando fuentes de datos diversas y metodologías avanzadas de análisis. Esto permite un monitoreo más eficiente, predicciones más precisas y una mejor adaptación a las necesidades específicas de las aplicaciones IoT en ambientes con recursos limitados.

11. MÉTRICAS PARA LA EVALUACIÓN DE LA PRECISIÓN Y EXACTITUD EN PREDICCIONES METEOROLÓGICAS

En el ámbito de la meteorología, la precisión y exactitud de las predicciones son fundamentales para la toma de decisiones informadas. Con el uso de LLMs como GPT-4, Claude, Llama, entre otros, surge la necesidad de evaluar su desempeño en comparación con los modelos convencionales utilizados en predicciones meteorológicas. Este capítulo se enfoca en identificar, seleccionar y validar un conjunto de métricas relevantes que permitan medir de manera cuantitativa la precisión y exactitud de las predicciones realizadas por LLMs frente a los modelos tradicionales.

11.1. MÉTRICAS CLÁSICAS PARA LA EVALUACIÓN DE MODELOS PREDICTIVOS

Para establecer una base sólida en la evaluación de modelos predictivos, es esencial considerar métricas ampliamente reconocidas en la literatura científica. Las siguientes métricas proporcionan una medida cuantitativa de la precisión del modelo y facilitan la identificación de posibles áreas de mejora.

11.1.1. ERROR CUADRÁTICO MEDIO (MSE)

El Error Cuadrático Medio es una métrica que cuantifica la diferencia promedio al cuadrado entre los valores predichos por el modelo y los valores reales observados. Matemáticamente, se define en la Ecuación 1:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Ecuación 1 Error cuadrático Medio (MSE).

donde:

- y_i representa el valor real.
- $\hat{y}_i$ es el valor predicho por el modelo.

- n es el número total de observaciones.

El MSE penaliza los errores grandes de manera más severa debido al término cuadrático, lo que lo hace especialmente útil cuando se desea evitar predicciones significativamente erróneas.

11.1.2. RAÍZ DEL ERROR CUADRÁTICO MEDIO (RMSE)

La Raíz del Error Cuadrático Medio (RMSE) es la raíz cuadrada del MSE. Se define en la Ecuación 2:

$$RMSE = \sqrt{MSE}$$

Ecuación 2 Raíz del Error Cuadrático Medio

El RMSE mantiene las mismas unidades que la variable de interés, facilitando su interpretación. Es particularmente útil para comparar la precisión de diferentes modelos en un mismo conjunto de datos.

11.1.3. COEFICIENTE DE DETERMINACIÓN (R^2)

El Coeficiente de Determinación, conocido como R^2 , mide la proporción de la variabilidad en los datos que es explicada por el modelo. Se calcula de la Ecuación 3:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Ecuación 3 Coeficiente de determinación R^2

Donde $\bar{y}$ es la media de los valores reales. Un R^2 cercano a 1 indica que el modelo explica una gran parte de la variabilidad de los datos, mientras que un valor cercano a 0 sugiere lo contrario.

11.2. MÉTRICA PARA MODELOS DE LENGUAJE DE GRAN TAMAÑO (LLMs)

Los LLMs presentan características únicas que requieren métricas adicionales para una evaluación más completa. Una métrica propuesta para este propósito es la *Probabilidad de Selección de Herramientas considerando la Longitud de Tokens*.

11.2.1. PROBABILIDAD DE SELECCIÓN DE HERRAMIENTAS

Integrar múltiples herramientas en un agente de modelo de lenguaje, como GPT-4, 3.5 Sonnet, entre otros, implica una consideración cuidadosa del límite de tokens del modelo. La métrica propuesta analiza la probabilidad de que un agente seleccione una herramienta específica, teniendo en cuenta la longitud de los tokens de las descripciones (docstrings) de cada una de las herramientas.

Análisis Matemático

Definamos las siguientes variables:

- T_{max} Límite máximo de tokens de la ventana de contexto del modelo (por ejemplo, 8192 tokens para GPT-4, 128k tokens GPT4o, 200k+ tokens Claude).
- $T_{context}$ Número de tokens utilizados por el resto del prompt (prompts del sistema, entrada del usuario, etc.).
- n Número total de herramientas disponibles.
- l_i Longitud en tokens de la descripción docstring para la herramienta.

La suma total de tokens consumidos por todas las descripciones de herramientas (docstrings) se muestra en la Ecuación 4:

$$T_{tools} = \sum_{i=1}^n L_i$$

Ecuación 4 Suma total de los tokens usados por docstring de las herramientas

Restricción del uso de tokens debido a su limite en la Ecuación 5

$$\sum_{i=1}^n L_i \leq T_{max} - T_{context}$$

Ecuación 5 Sumatoria de token considerando la restricción de token por modelo y contexto

Probabilidad de selección de herramientas

La capacidad del agente para seleccionar una herramienta depende de

1. Inclusión en el contexto: una herramienta debe estar incluida dentro del límite de tokens para ser considerada.
2. Frecuencia de uso: datos de uso históricos que indican la frecuencia con la que se utiliza cada herramienta.

Probabilidad de frecuencia de uso

Según el uso histórico *usage*, la probabilidad de usar la herramienta *i* se define en la Ecuación 6

$$P_{usage,i} = \frac{N_{uso,i}}{N_{total}}$$

Ecuación 6 Probabilidad de uso de la herramienta por parte del LLM

Donde:

- $N_{uso,i}$ Número de veces que se utilizó la herramienta *i*
- N_{total} : Número total de usos de la herramienta.

Ejemplo Usando GPT4: , Para obtener el total de tokens utilizados por los docstrings de cada herramienta, se empleó la herramienta de tokenización de OpenAI, disponible en <https://platform.openai.com/tokenizer>, Figura 10. Los modelos de lenguaje de OpenAI procesan texto usando tokens, los cuales representan secuencias comunes de caracteres. Estos modelos aprenden a entender las relaciones estadísticas entre dichos tokens y se destacan en predecir el siguiente token en una secuencia.

- $T_{max} = 8192 \text{ tokens}$
- $T_{context} = 2000 \text{ tokens}$
- *Tools*: 7 herramientas con las siguientes longitudes de token y probabilidades de uso.

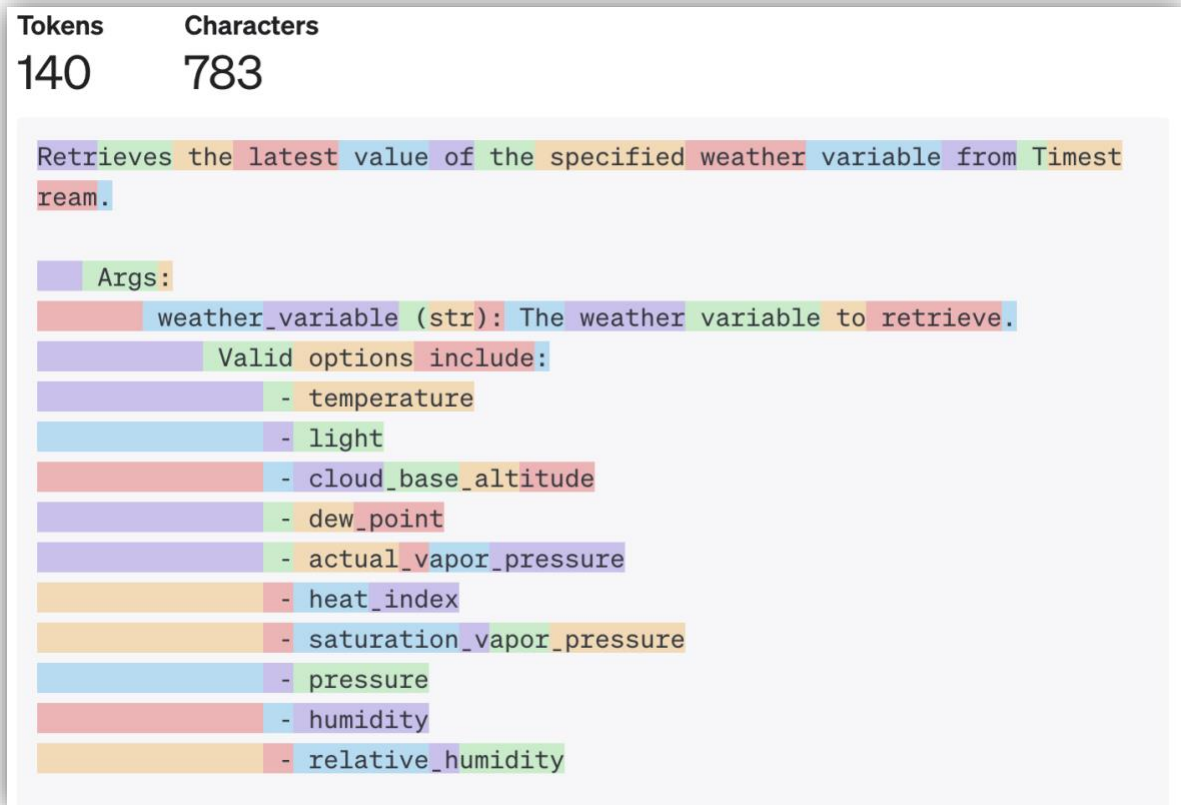


Figura 10 Cantidad de tokens utilizados por la herramienta *get_weather* usando tokenizer de OpenAI. Fuente: Autores

En la siguiente Tabla 1 se muestra el total de tokens utilizados por dos strings

Tabla 1 Muestra el total de tokens por cada una de las herramientas creadas para el Agente: Fuente: Autores

Tool	l_i (tokens)
get_weather	140
get_communication	123

get_current_time	80
load_model_and_predict	153
evaluate_model_performance	197
detect_anomalies_from_iot_sensor	91
get_weather_openweathermap	120
Total	904

Total de tokens todas las herramientas:

$$T_{tools} = 904$$

Obteniendo el total de tokens:

$$T_{context} + T_{tools} = 2904 \text{ tokens}$$

Tokens aproximados disponibles para herramientas:

$$8192 \text{ tokens} - 2904 \text{ tokens} = 5288 \text{ tokens}$$

11.2.2. MÉTRICAS DE EFICIENCIA OPERACIONAL

Además de la probabilidad de selección de herramientas, es crucial evaluar la eficiencia operacional de los agentes LLM. Las siguientes métricas son necesarias para esta evaluación, ya que proporcionan una visión integral del desempeño del modelo en términos de recursos y costos asociados.

Latencia: Se refiere al tiempo total que tarda el modelo en procesar una solicitud y generar una respuesta. Es una métrica esencial, especialmente en aplicaciones en tiempo real donde la rapidez de la respuesta es crítica. Una baja latencia garantiza una experiencia de usuario fluida y eficiente, mientras que una alta latencia puede resultar en demoras inaceptables y disminuir la utilidad del modelo en entornos dinámicos

Costo: El costo asociado a los agentes ReAct en LLMs es una consideración crucial que se deriva principalmente del uso de tokens en las respuestas y los procesos de razonamiento. Los agentes ReAct están diseñados para combinar capacidades de

razonamiento y acción, lo que implica múltiples interacciones con el modelo para generar respuestas más complejas y contextualizadas.

Tokens Usados: El número de tokens usados por el modelo en cada predicción es una métrica clave que afecta tanto la eficiencia computacional como los costos operacionales. Los tokens representan las unidades básicas de procesamiento del modelo, y un uso eficiente de ellos puede llevar a mejoras significativas en la velocidad de generación de respuestas y en la reducción de costos asociados al procesamiento de grandes volúmenes de datos. Optimizar el uso de tokens implica desarrollar estrategias que minimicen el consumo sin comprometer la calidad de las predicciones (Brown et al., 2020).

11.2.3. MÉTRICAS CUALITATIVAS DE EVALUACIÓN ESPECÍFICAS PARA AGENTES LLM

Además de las métricas cuantitativas, es importante considerar criterios cualitativos que evalúen aspectos fundamentales del desempeño de los agentes LLM. Estos criterios proporcionan una visión más completa de la calidad de las predicciones y la interacción del modelo con los usuarios, permitiendo una evaluación detallada de su efectividad y fiabilidad.

Calidad de la Respuesta: Garantizar respuestas de alta calidad es vital para la efectividad de la aplicación IoT. Respuestas de baja calidad pueden llevar a acciones incorrectas o interpretaciones erróneas, afectando la fiabilidad del sistema.

Exactitud de la Información: Este criterio evalúa si la predicción realizada por el modelo coincide con el valor real observado. Es una medida directa de la precisión del modelo y fundamental para garantizar la confiabilidad de las predicciones meteorológicas.

Transparencia en el Razonamiento: Este criterio evalúa la capacidad del modelo para comunicar de manera transparente cómo se derivó la predicción, lo que es esencial para la confianza y la interpretabilidad del usuario.

Calidad del Razonamiento: La capacidad del LLM para realizar razonamientos lógicos y proporcionar respuestas coherentes es esencial para procesos de toma de decisiones complejos dentro de los sistemas IoT. Una alta calidad de razonamiento mejora la capacidad del sistema para manejar escenarios diversos e imprevistos.

Apropiado uso de Herramientas: Este criterio verifica si el modelo emplea correctamente las herramientas disponibles para obtener información precisa, asegurando una integración eficaz de recursos externos en el proceso de predicción.

Complejidad de la Respuesta: Evaluar la complejidad asegura que el modelo aborde todos los aspectos relevantes de la consulta sin omitir información crucial, proporcionando una respuesta integral al usuario.

Consistencia Lógica: Este criterio garantiza que las inferencias del modelo sean coherentes y no presenten inconsistencias internas, lo que es vital para mantener la integridad de las predicciones.

Manejo de Ambigüedades: Evaluar el manejo de ambigüedades es fundamental para determinar la capacidad del modelo para interpretar y resolver preguntas que no son completamente claras, mejorando así la robustez de las respuestas.

Eficiencia en la Resolución de Problemas: Este criterio mide la capacidad del modelo para proporcionar soluciones óptimas sin incurrir en redundancias o procesos ineficientes, lo que contribuye a una mayor eficacia en la generación de respuestas.

Desempeño General: Este criterio proporciona una evaluación holística del desempeño del modelo, integrando todos los aspectos anteriores para una valoración global de su efectividad y fiabilidad.

Número de Herramientas Utilizadas: La cantidad de herramientas o recursos externos que un LLM utiliza, según lo decidido por el agente ReAct, puede impactar su versatilidad y adaptabilidad. Un mayor número de herramientas puede mejorar la funcionalidad pero también aumentar la complejidad y los requisitos de recursos.

12. SELECCIÓN DE LOS LLMs A EVALUAR EN SISTEMAS DE MONITOREO METEOROLÓGICO IOT

En este capítulo se describen los LLMs seleccionados para la evaluación en el contexto de un sistema de monitoreo meteorológico IoT. Estos modelos fueron seleccionados debido a sus diferentes capacidades de comprensión del lenguaje natural y su potencial aplicabilidad en la interpretación y análisis de datos ambientales generados por dispositivos IoT. Se hace necesario mencionar que dichos modelos fueron seleccionados y posteriormente evaluados en Junio de 2024. A continuación, se describen los modelos utilizados y se realiza una comparación entre ellos.

12.1. CLAUDE-3-5-SONNET-20240620 (ANTHROPIC)

Claude 3.5 Sonnet es el modelo de lenguaje más avanzado desarrollado por Anthropic. Destaca en tareas de comprensión del lenguaje natural, razonamiento y programación. Este modelo se caracteriza por sus significativas mejoras en velocidad y eficiencia de costos en comparación con sus versiones anteriores. Está diseñado para manejar aplicaciones complejas, como soporte al cliente y automatización de flujos de trabajo. Claude 3.5 Sonnet demuestra una gran capacidad para razonar de manera coherente, lo cual es crucial para entornos de IoT donde la toma de decisiones en tiempo real es fundamental.

12.2. GPT-4O (OPENAI)

GPT-4o, desarrollado por OpenAI, es la última iteración de la serie GPT-4. Este modelo está enfocado en mejorar la comprensión del lenguaje, la generación de contenido y la comprensión contextual. Es altamente eficaz para una variedad de tareas conversacionales y analíticas. Su capacidad para interpretar datos, generar respuestas naturales y adaptarse al contexto hace que sea ideal para integrarse en sistemas IoT, donde la interacción con el usuario y la explicación de resultados juegan un papel importante.

12.3. LLAMA-V3-70B-INSTRUCT (META AI)

LLaMA v3-70B es parte de la serie de modelos de lenguaje de Meta AI, diseñado para aplicaciones a gran escala. Este modelo está optimizado para tareas de seguimiento de instrucciones, proporcionando respuestas detalladas y precisas. Sus capacidades de comprensión y generación de lenguaje lo hacen relevante para aplicaciones que requieren un alto nivel de precisión en la interpretación de información. En el contexto de sistemas IoT, LLaMA es adecuado para procesar instrucciones complejas y proporcionar explicaciones detalladas de datos técnicos.

12.4. YI-LARGE (01.AI)

Yi-Large, desarrollado por 01.AI, se enfoca en proporcionar un rendimiento robusto en comprensión y generación del lenguaje natural. Diseñado para manejar desde consultas simples hasta interacciones más complejas, Yi-Large es versátil y eficiente en diversas aplicaciones. Esta versatilidad lo convierte en una opción interesante para sistemas IoT, ya que puede adaptarse a distintas necesidades de interpretación y análisis de datos, asegurando una solución ágil y eficaz.

13. IMPORTANCIA DE LA OBSERVABILIDAD EN LLMS

La observabilidad y el monitoreo son componentes esenciales en el desarrollo y gestión de LLMS, particularmente cuando estos se integran en sistemas complejos como los sistemas de monitoreo meteorológico IoT. LangSmith permite a los desarrolladores obtener una visión profunda del funcionamiento interno de los LLMS, lo cual es crucial para garantizar el rendimiento, la precisión y la confiabilidad del modelo (Langsmith, 2024).

La observabilidad proporciona la capacidad de inspeccionar los estados internos del modelo, lo cual facilita la detección temprana de problemas, como sesgos, imprecisiones o comportamientos inesperados. Esto es especialmente importante en aplicaciones que requieren respuestas precisas, como el monitoreo de condiciones ambientales en tiempo real. La observabilidad ayuda a comprender la razón detrás de las decisiones tomadas por los LLMS, mejorando la capacidad para ajustar el sistema de manera que se eviten errores recurrentes o fallas sistémicas, la interfaz se muestra en la Figura 11.

The screenshot displays the LangChainsmith web interface for a project named "langgraph-react-iot-test". The interface is divided into several sections:

- Runs Table:** A table listing various runs with columns for Name, Input, and Output. The runs include tasks like "evaluate_model_perform", "load_model_and_predic", "get_weather" (for light, pressure, humidity, and temperature), "get_weather_openweatl", and "LangGraph".
- Trace View:** A detailed view of a specific run, showing a sequence of steps and their durations. The steps include "agent_reason" (1.80s), "PromptTem..." (0.00s), "seq.step:2", "ChatOpenAI gpt-4o" (1.78s, seq.step:3), "should_continue" (0.00s, seq.step:3), "act" (1.79s, graph.step:2), and another "agent_reason" (0.97s).
- Input/Output:** The input for the selected run is "input: Please get current temperature, predict the temperature in the next hour and evaluate the model's performance and decide if better to use OpenWeatherMap". The rendered output shows the agent's reasoning and the final output: "The current temperature from Timestream is 23.92°C, and the predicted temperature for the next hour is 22.32°C. The model's performance metrics show some uncertainty. The current temperature from OpenWeatherMap is 14.96°C. Given the discrepancies, it might be better to use".
- Metadata:** Key performance indicators for the run, including START TIME (10/28/2024, 09:44:19 PM), END TIME (10/28/2024, 09:44:53 PM), TIME TO FIRST TOKEN (N/A), STATUS (Success), TOTAL TOKENS (13,152 tokens / \$0.07258), LATENCY (34.08s), and TYPE (Chain).

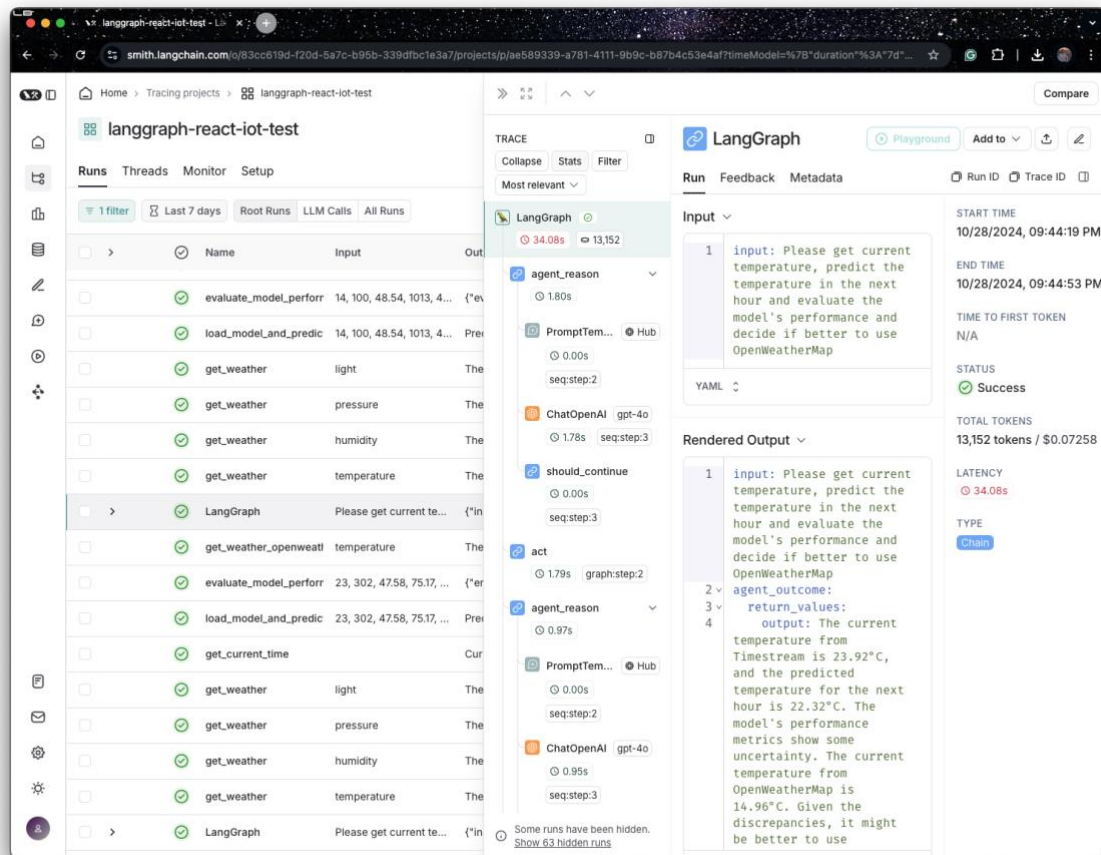


Figura 11 Interfaz de LangSmith, se muestra diferentes herramientas usadas por el agente ReAct.
Fuente: Autores

Por otro lado, el monitoreo implica un seguimiento continuo de las métricas de rendimiento del modelo en tiempo real. Esto permite que se cumplan los estándares deseados y que cualquier desviación sea identificada de manera oportuna. Aspectos como la latencia, el uso de recursos (tokens) y la calidad de las respuestas deben monitorearse constantemente para mantener la eficiencia operativa del sistema IoT(symflower, 2024).

13.1. BENEFICIOS DEL USO DE LANGSMITH

LangSmith facilita la visualización y el análisis de múltiples aspectos del rendimiento de los modelos evaluados, proporcionando herramientas que permiten:

- Monitoreo en tiempo real de métricas críticas, tales como latencia, número de tokens utilizados y calidad de las respuestas.
- Identificación de patrones o tendencias en el comportamiento del modelo, lo cual es útil para ajustar el rendimiento según la evolución del entorno o las necesidades del sistema IoT.
- Evaluación de la eficiencia del modelo en el uso de herramientas y recursos, minimizando costos y optimizando la funcionalidad en un contexto de implementación a gran escala.

LangSmith es particularmente útil para garantizar que el sistema se mantenga confiable en condiciones cambiantes, lo cual es fundamental para un sistema de monitoreo meteorológico que depende de la precisión y rapidez de respuesta.

14. PREDICCIÓN DE VARIABLES METEOROLÓGICAS MEDIANTE RANDOM FOREST REGRESSION

Este capítulo aborda la selección e implementación del modelo Random Forest Regression para predecir variables meteorológicas, utilizando datos almacenados en AWS Timestream y posteriormente procesados por un Agente React.

14.1. MODELOS DE REGRESIÓN EN PREDICCIÓN METEOROLÓGICA

La predicción meteorológica se sustenta en modelos matemáticos y estadísticos que analizan patrones históricos para prever condiciones futuras. Entre los métodos más utilizados se encuentran los modelos de regresión, que establecen relaciones entre variables dependientes e independientes. Estos modelos son particularmente útiles para predecir variables continuas como la temperatura, la humedad y la presión atmosférica.

Según Hastie, Tibshirani y Friedman, los modelos de regresión ofrecen una base sólida para entender las relaciones subyacentes entre variables y permiten realizar predicciones precisas cuando se cuenta con datos de calidad y se elige el modelo adecuado (Hastie et al., 2009).

14.2. RANDOM FOREST REGRESSION: FUNDAMENTOS Y VENTAJAS

El Random Forest Regression es un algoritmo de aprendizaje supervisado basado en ensamblajes de árboles de decisión. Introducido por Breiman (2001), este método combina múltiples árboles de decisión para mejorar la precisión predictiva y reducir el riesgo de sobreajuste. Cada árbol en el bosque se entrena con una muestra aleatoria de los datos y una selección aleatoria de características, lo que introduce diversidad y mejora la generalización del modelo (Breiman, 2001).

Entre las principales ventajas del Random Forest Regression se encuentran:

- **Robustez frente a Datos Ruido y Outliers:** La agregación de múltiples árboles mitiga el impacto de anomalías en los datos.

- **Capacidad de Manejar Grandes Conjuntos de Datos:** Es escalable y eficiente en términos computacionales.
- **Evaluación de la Importancia de Variables:** Permite identificar las características más relevantes para la predicción, lo que es crucial para la interpretación de los resultados.

Entrenamiento del Modelo Random Forest Regression

El modelo Random Forest Regression se entrena utilizando el conjunto de datos preprocesado. Se seleccionan características relevantes como la hora, el día del año, la humedad, la presión y la luminosidad como variables independientes, mientras que la temperatura actúa como variable dependiente. La división de los datos en conjuntos de entrenamiento y prueba permite evaluar la capacidad predictiva del modelo.

El entrenamiento inicial se realiza con parámetros estándar, y posteriormente se lleva a cabo una optimización de hiperparámetros mediante técnicas como GridSearchCV, que exploran diferentes combinaciones de parámetros para identificar la configuración que minimiza el error cuadrático medio (Pedregosa FABIANPEDREGOSA et al., 2011).

Evaluación del Modelo

La evaluación del modelo se realiza utilizando métricas como el Error Cuadrático Medio (MSE), la Raíz del Error Cuadrático Medio (RMSE) y el Coeficiente de Determinación (R^2). Estas métricas proporcionan una medida cuantitativa de la precisión del modelo y su capacidad para generalizar a nuevos datos.

Además, se emplean técnicas de validación cruzada para asegurar la robustez del modelo frente a diferentes particiones de los datos.

Implementación y Uso por el Agente React

Una vez entrenado y evaluado, el modelo se almacena en AWS S3 para facilitar su acceso y despliegue. El Agente React, una aplicación desarrollada para interactuar con el modelo, recupera las predicciones en tiempo real y las presenta de manera intuitiva a los usuarios.

finales. Esta integración permite una visualización dinámica de las predicciones meteorológicas, mejorando la toma de decisiones basada en datos precisos y actualizados.

15. METODOLOGÍA

La metodología propuesta consta de cinco etapas iterativas fundamentales: Identificación, Desarrollo, Prueba y Monitoreo, Evaluación y Análisis. Cada una de estas etapas desempeña un papel importante en la garantía de una evaluación exhaustiva, así como en la facilitación de la mejora continua, Figura 12.

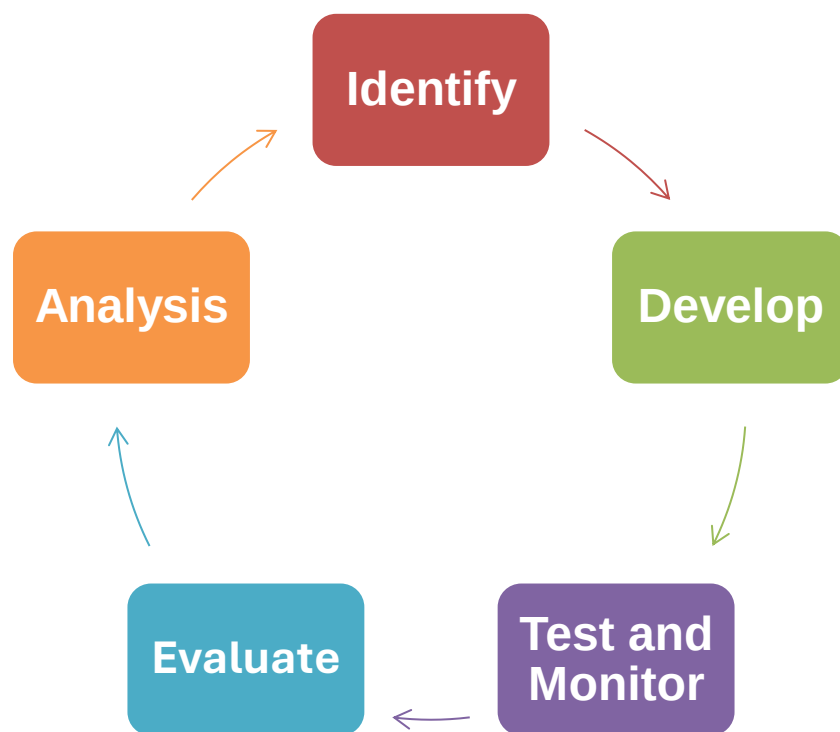


Figura 12 Metodología para la selección de LLMs. Fuente: Autores

15.1. IDENTIFICACIÓN

La etapa de Identificación comienza con la definición clara de la aplicación IoT específica y el papel del modelo de lenguaje de gran tamaño en el sistema. Esto implica la selección de las métricas de rendimiento más relevantes, tales como latencia, precisión, costo, eficiencia energética, seguridad entre otros, que se alineen con los objetivos de la aplicación.

Además, se preparan las pruebas necesarias para medir estas métricas de manera efectiva, estableciendo así un plan de evaluación. La selección del LLM más adecuado implica considerar diversos modelos y evaluar su impacto potencial sobre el sistema IoT.

15.2. DESARROLLO

Durante la etapa de Desarrollo, se introducen modificaciones al LLM o al sistema IoT con el objetivo de facilitar la monitoreo y la recolección de datos. Esto incluye la configuración de herramientas de monitoreo y la integración de mecanismos de registro para la captura precisa de las métricas seleccionadas. El objetivo es dotar al sistema de las herramientas necesarias para proporcionar datos detallados sobre el rendimiento durante la fase de pruebas. También puede ser necesaria una configuración adicional que asegure el funcionamiento eficiente del LLM dentro de las restricciones de recursos de los dispositivos IoT.

15.3. PRUEBA Y MONITOREO

En la etapa de prueba y monitoreo se ejecutan las pruebas preparadas previamente, en escenarios y condiciones definidos durante la fase de Identificación. El LLM se somete a pruebas tanto en entornos simulados como reales para la recopilación de una amplia gama de datos sobre su rendimiento. Asimismo, se prueban distintos LLMs para comparar su comportamiento y determinar el más adecuado para la aplicación IoT. La recolección continua de datos relativos a las métricas seleccionadas se realiza mediante herramientas de monitoreo configuradas. Esta etapa proporciona información actualizada sobre el rendimiento del sistema bajo diversas condiciones y contribuye a la comprensión de su comportamiento operativo. La monitoreo en tiempo real permite identificar problemas o degradaciones de rendimiento que puedan surgir durante su uso prolongado. Los resultados de estas pruebas se registran de manera meticulosa para proporcionar una base robusta que facilite el análisis subsecuente.

15.4. EVALUACIÓN

La etapa de Evaluación se evalúan los LLMs mediante técnicas automáticas y evaluaciones humanas. Las técnicas automáticas comprenden el uso otros LLMs que comparan las

respuesta con la deseada para obtener resultados cuantitativos y la efectividad general del modelo. Por su parte, la evaluación humana valora la creatividad, el estilo, el tono y la relevancia, mediante calificación binaria (pulgar arriba/abajo) y una escala de cinco puntos. Este enfoque dual garantiza una evaluación equilibrada y profunda de los modelos, considerando tanto las métricas objetivas de rendimiento como el juicio subjetivo humano.

15.5. ANÁLISIS

Durante la etapa de Análisis, se examinan a fondo los datos recogidos en todas las etapas anteriores para identificar tendencias, anomalías y áreas de mejora. Se realizan análisis tanto estadísticos como cualitativos con el fin de extraer conclusiones significativas sobre el rendimiento del LLM. En base a estos hallazgos, se recomiendan acciones para la selección del modelo. En caso de ser necesario, se planifican pruebas adicionales, lo que da lugar a un nuevo ciclo de la metodología orientado hacia la mejora continua. El objetivo final es refinar la selección del LLM que mejor se ajuste a las necesidades específicas de la aplicación IoT.

16. CASO DE ESTUDIO: MEDICIÓN DE VARIABLES CLIMATICAS EN HOME OFFICE

En el presente capítulo se aplicará la metodología propuesta en un caso de estudio real, cuyo objetivo es medir las variables meteorológicas dentro de un home office mediante sensores IoT para monitorear temperatura, humedad, iluminación, entre otras; con la integración de los modelos de lenguajes de gran tamaño se quiere mejorar la exactitud y precisión de la predicción de las variables climáticas y medir su respectivo impacto.

16.1. IDENTIFICACIÓN

16.1.1. COMPONENTES

En la etapa de identificación, el sistema se compone de un módulo con sensores para la medición de variables climáticas y un Gateway LoRaWAN, el cual se ha registrado en AWS para transmitir la información generada por el módulo sensor.

Módulo LoRaWAN: Utiliza el protocolo LoRaWan® 1.0.2 (RAK19007 / RAK4631) con sensores integrados, como se muestra en la parte inferior de la Figura 13. Este módulo ha sido programado no solo para transmitir los valores de los sensores, sino también para enviar información sobre el estado de la comunicación. Los sensores climáticos integrados son:

- Sensor de temperatura y humedad (RAK1901)
- Sensor de presión barométrica (RAK1902)
- Sensor de luz (RAK1903)

Gateway: Se utiliza un Gateway LoRaWAN de 8 canales para interiores (RAK7268), el cual se ha configurado y registrado en AWS mediante credenciales de seguridad. Este Gateway permite la comunicación entre los sensores y los servicios de nube mediante el uso de redes LPWAN (Low-Power Wide-Area Network) como se muestra en la parte superior de la Figura

13, que son ideales para aplicaciones IoT gracias a su bajo consumo de energía y su capacidad para transmitir datos a largas distancias (Sornin et al., 2015)

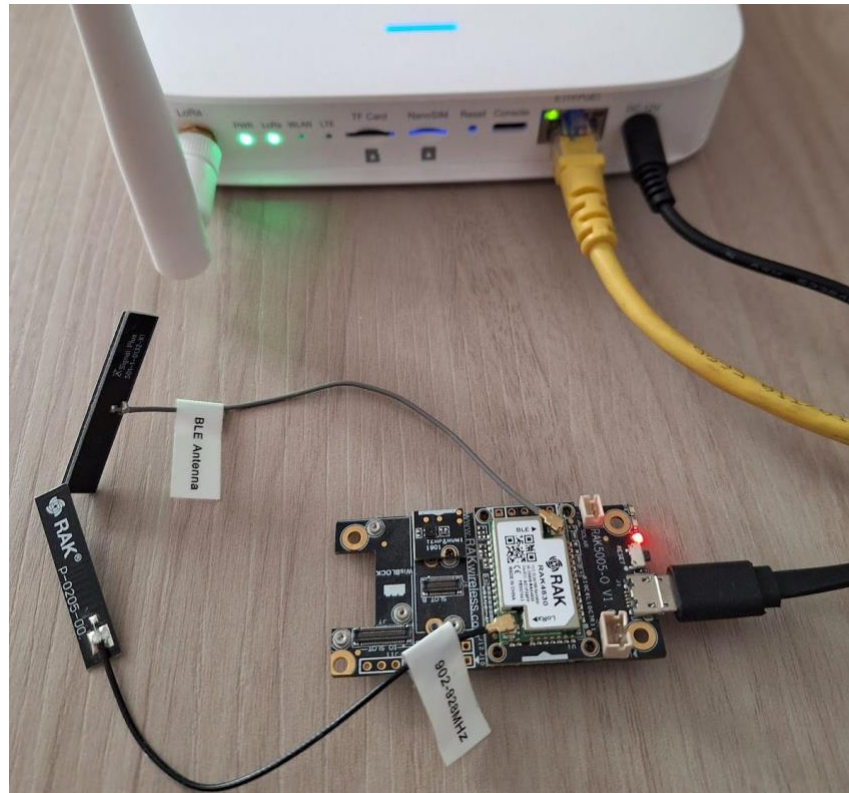


Figura 13 Gateway RAK7268, modulo RAK19007 / RAK4631. Fuente: Autores

AWS IoT Core: Es un servicio administrado que permite conectar de forma segura dispositivos IoT a la nube. Se encarga de recibir la información enviada por el Gateway, administrar las conexiones, y activar reglas o desencadenar funciones de manera eficiente. En este caso, AWS IoT Core recibe la información y activa una regla para ejecutar una función Lambda, facilitando la integración entre dispositivos y la nube (AWS, 2024b).

AWS Lambda: Es un servicio que permite ejecutar código sin necesidad de aprovisionar ni gestionar servidores. En este sistema, Lambda se encarga de recibir, decodificar y procesar la información enviada por los sensores y almacenarla en Amazon Timestream. La capacidad de procesamiento bajo demanda de Lambda permite una respuesta ágil y escalable sin preocuparse por la infraestructura (AWS, 2024c)

Amazon Timestream: Es una base de datos optimizada para el almacenamiento de series de tiempo, ideal para aplicaciones IoT que generan grandes volúmenes de datos con marcas de tiempo. En este caso, se utiliza para almacenar los datos enviados por el sensor, incluyendo los valores climáticos y de comunicación, junto con sus respectivas marcas de tiempo(AWS, 2024a).

LangGraph y Agente ReAct: El código se desarrolla en Python utilizando la librería LangGraph para configurar un agente ReAct, el cual cuenta con herramientas para obtener información de la base de datos Amazon Timestream. Aquí se gestiona la comunicación y el uso de diferentes LLM, LangGraph proporciona un marco para representar y analizar la información de manera eficiente, lo cual es fundamental en aplicaciones de IA que requieren razonamiento avanzado.

LangSmith: LangSmith es un framework de observabilidad utilizado para evaluar el rendimiento de los diferentes modelos de LLM en términos de latencia, análisis de flujos, uso de tokens, entre otros. Este framework permite entender cómo interactúan los LLMs con otras herramientas y medir su eficiencia en tiempo real.

Grafana: Para la visualización de los datos, se ha utilizado Grafana en conjunto con el servicio ECS de AWS. Grafana es una plataforma de análisis y monitoreo que permite la creación de paneles dinámicos para visualizar métricas en tiempo real, facilitando el análisis del comportamiento de los datos generados por el sistema IoT, en la Figura 14 se muestra las variables de comunicación del sensor que pueden ser visualizadas en el dashboard. En la Figura 15 se muestra dashboard de grafana sobre las variables meteorológicas del sensor.



Figura 14 dashboard que muestra las variables de comunicación del sensor. Hecho con Grafana. Fuente Autores



Figura 15 Dashboard creado para la visualización de los datos capturados por el sensor IoT, hecho con Grafana. Fuente Autores

16.1.2. ARQUITECTURA

La arquitectura diseñada para la plataforma en la nube está orientada a proporcionar un almacenamiento escalable y eficiente para las cargas de datos generadas por dispositivos IoT. Esta arquitectura es de tipo serverless y basada en eventos (event-driven), lo cual permite una gestión flexible y adaptable a las demandas cambiantes de los dispositivos conectados. Los principales componentes de esta arquitectura se dividen en cuatro etapas:

ingestión, almacenamiento, visualización y análisis de la información, la arquitectura descrita se muestra en la Figura 16.

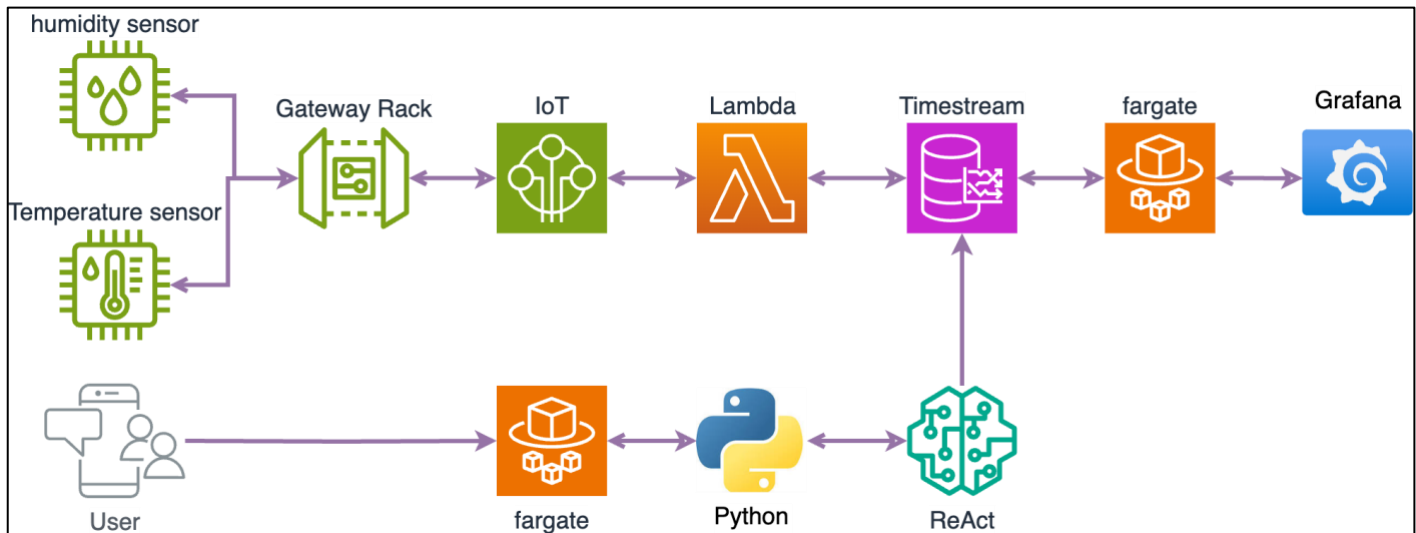


Figura 16 Arquitectura serverless para aplicación de metodología para la selección de LLMs en aplicaciones de IoT.
Fuente: Autores

A continuación, se explica cada una de las etapas del sistema IoT, en la Figura 17 se muestra el orden proceso.

Etapa de Ingestión

La etapa de ingestión está compuesta por los sensores que recolectan datos de variables climáticas y los envían a un Gateway LoRaWAN u otro dispositivo que permita la recepción de estos datos. El Gateway debe configurarse para ser identificado y autenticado en AWS IoT Core, asegurando una conexión segura y confiable. Esta infraestructura permite agregar múltiples sensores y gateways al servicio de AWS de manera escalable, facilitando la expansión sin interrupciones conforme se incrementa el número de dispositivos.

Cada vez que un dato es recibido por AWS IoT Core, se activa una regla de AWS IoT (AWS IoT Rule) que invoca una función AWS Lambda. Este proceso invoca la ejecución de una función Lambda de forma paralela por cada dato recibido, lo cual permite escalar a cientos de invocaciones simultáneas sin afectar el rendimiento. AWS Lambda se encarga de decodificar la información enviada por los sensores y de almacenar los datos procesados

en el sistema de almacenamiento correspondiente, garantizando una gestión eficiente y dinámica de los datos ingresados.

Etapas de Almacenamiento

Para la etapa de almacenamiento se utiliza Amazon Timestream, un servicio especializado en la gestión y almacenamiento eficiente de datos de series temporales provenientes de los sensores, recibidos a través de AWS IoT Core. Amazon Timestream está optimizado para manejar grandes volúmenes de datos de IoT y facilitar análisis en tiempo real, ofreciendo funcionalidades como almacenamiento optimizado para series temporales que permiten la consulta y análisis de datos basados en el tiempo de manera eficiente.

Además, Amazon Timestream es un servicio completamente gestionado y serverless, lo que significa que no es necesario administrar la infraestructura subyacente y que permite una escalabilidad automática ajustándose dinámicamente al volumen de datos entrantes. Este servicio también implementa almacenamiento jerárquico (tiered storage), donde los datos recientes se almacenan en memoria para un acceso más rápido, mientras que los datos históricos se trasladan automáticamente a un almacenamiento más económico. Las capacidades de agregación integradas y las funciones analíticas de Timestream permiten realizar consultas complejas de manera rápida y eficiente (AWS, 2024a)

Etapas de Visualización

En la etapa de visualización se implementa un dashboard en Grafana, desplegado mediante Amazon Elastic Container Service (ECS) utilizando AWS Fargate. Esta configuración facilita la visualización eficiente y escalable de los datos de los sensores, que incluyen tanto variables ambientales como información de comunicación. El dashboard de Grafana se integra directamente con Amazon Timestream mediante un plugin, lo que permite el acceso y la visualización en tiempo real de los datos almacenados.

Para simplificar la configuración y gestión, se utiliza Amazon Elastic File System (EFS) para almacenar la información y configuraciones de los dashboards de Grafana. Esta arquitectura permite crear y desplegar tareas en ECS únicamente cuando se necesita visualizar los datos, y detenerlas cuando no están en uso, optimizando así el uso de recursos y reduciendo los costos operativos. Además, esta arquitectura serverless

garantiza una alta disponibilidad y flexibilidad, permitiendo que la infraestructura de visualización escale según la demanda sin necesidad de intervención manual. En conjunto, estas soluciones aseguran una experiencia de monitoreo robusta y rentable, facilitando el análisis y la toma de decisiones basadas en los datos de los sensores (Grafana, 2024).

Etapas de Análisis de Información

En la etapa de análisis de información se emplean LLMs para interpretar y extraer significado de los datos recolectados, facilitando la identificación de tendencias, patrones y resúmenes de hallazgos relevantes. Este proceso se implementa mediante un código en Python que configura un agente tipo ReAct, el cual combina capacidades de razonamiento y acción para interactuar de manera eficiente con los datos utilizando herramientas que acceden a la información recopilada por los sensores IoT.

Mediante uso de procesamiento de lenguaje natural y análisis de datos, esta etapa no solo optimiza la comprensión de grandes volúmenes de información, sino que también potencia la toma de decisiones informadas y estratégicas. Además, la automatización proporcionada por los LLMs asegura una respuesta rápida y adaptativa a las variaciones detectadas por los sensores, mejorando así la eficiencia operativa y la capacidad de reacción ante diferentes escenarios. Esta capacidad es esencial para entornos IoT donde la rapidez y la adaptabilidad son cruciales para garantizar un funcionamiento óptimo del sistema.



Figura 17 Etapas de la plataforma para aplicación de metodología. Fuente Autores

16.1.3. EXPLICACIÓN DEL FLUJO DEL SISTEMA

La explicación del flujo e interacción de los componentes se muestra en la Figura 18..

- 1. Envío de Información a AWS:** El módulo IoT, equipado con diversos sensores climáticos, envía tramas codificadas al Gateway RAK. Este Gateway se ha configurado y registrado en AWS utilizando credenciales de seguridad. Dentro de AWS IoT Core, se ha creado una regla que activa una función Lambda. Esta función Lambda decodifica las tramas enviadas por el sensor y almacena los valores correspondientes a las variables climáticas y de comunicación en Amazon Timestream, añadiendo la marca de tiempo correspondiente.
- 2. Interacción del Agente ReAct:** En esta etapa, el agente ReAct, utilizando la librería LangGraph y haciendo uso de un modelo LLM de prueba, se comunica con Amazon Timestream mediante la librería Boto para obtener información de los sensores y del clima.
- 3. Monitoreo con LangSmith:** Se utiliza LangSmith para monitorear el procesamiento y el flujo de interacción del agente con el modelo LLM, así como el uso de herramientas. Aquí se obtiene información relevante, como la cantidad de tokens usados y la latencia en la ejecución.

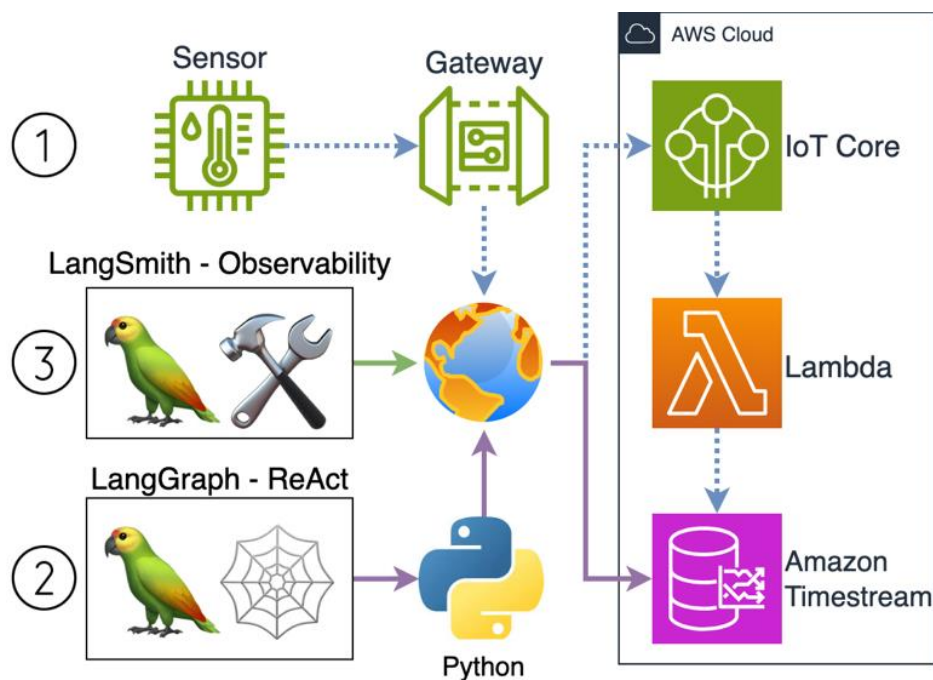


Figura 18 flujo e interacción de los componentes. Fuente Autores

16.2. DESARROLLO

En esta etapa del caso de estudio se describe el proceso de desarrollo, en el cual se seleccionan las herramientas adecuadas para el monitoreo y la aplicabilidad de la metodología para el sistema de monitoreo meteorológico basado en IoT. La herramienta LangSmith fue seleccionada para esta fase debido a sus capacidades para mejorar el desarrollo, despliegue y mantenimiento de los LLMs. Por lo tanto, se importaron la librerías necesarias para su uso y se utilizó el Api key de langSmith para poder proceder con la parte de monitoreo de la metodología.

A continuación, se muestra una imagen de la interfaz de LangSmith en la Figura 19

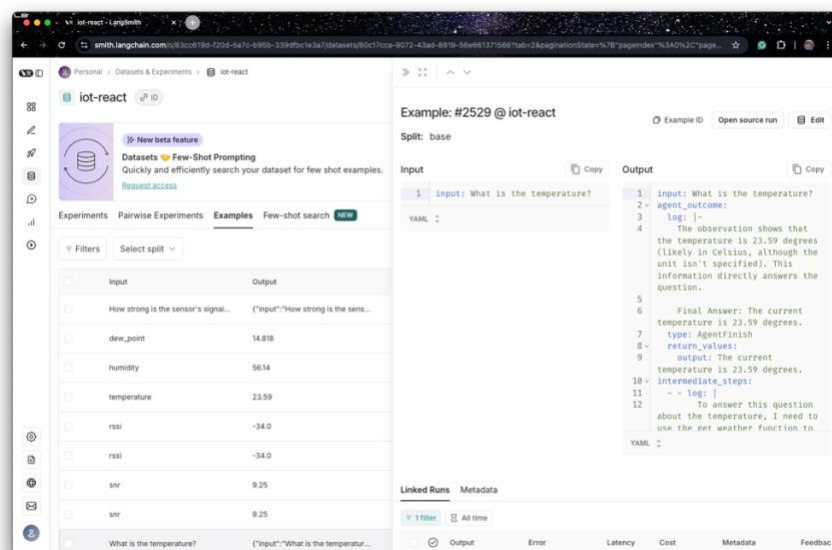


Figura 19 Interfaz de LangSmith plataforma usada para el monitoreo. Fuente: Autores

La imagen adjunta muestra la interfaz del proyecto dentro de LangSmith. A continuación, se explican los puntos más importantes de la información presentada:

- **Run Count:** Indica la cantidad de ejecuciones realizadas del modelo durante el período especificado (últimos 7 días).

- **Total Tokens:** Representa el número total de tokens utilizados por el modelo y el costo asociado.
- **Latencia:** La métrica de latencia es crucial para determinar la rapidez con la que el modelo responde a las solicitudes. LangSmith permite filtrar las ejecuciones según la latencia, por ejemplo, para identificar aquellas con latencia mayor o igual a 10 segundos.
- **Detalles de las Ejecuciones:** En la interfaz se presentan columnas que incluyen información como el nombre de la ejecución, el error (si aplica), la hora de inicio, la latencia, el uso de tokens, el costo, entre otros. Esta información es fundamental para evaluar y optimizar el rendimiento del modelo.
- **Filtros:** La herramienta ofrece opciones de filtro, como ver las ejecuciones que fueron canceladas o aquellas con alta latencia, lo cual es útil para identificar y abordar problemas específicos del modelo.

La interfaz de LangSmith proporciona una visión clara y estructurada de las métricas esenciales para la evaluación de los LLMs, lo cual facilita la toma de decisiones basadas en datos y la mejora continua del sistema.

16.3. PRUEBAS Y MONITOREO

Para llevar a cabo la evaluación de los diferentes modelos de lenguaje, se diseñaron una serie de preguntas orientadas a medir tanto la calidad de las respuestas como la capacidad de razonamiento, dado que se utiliza un agente del tipo ReAct. Estas preguntas están estructuradas en distintos niveles de complejidad para evaluar de manera integral el desempeño de cada modelo en aplicaciones IoT.

16.3.1. SELECCIÓN DE PREGUNTAS DE EVALUACIÓN

La selección de preguntas se realizó con el objetivo de abarcar diferentes aspectos y niveles de complejidad, asegurando una evaluación exhaustiva de las capacidades del modelo. Las preguntas seleccionadas son las siguientes, Tabla 2 Relación de preguntas realizadas al agente y nivel de complejidad:

Tabla 2 Relación de preguntas realizadas al agente y nivel de complejidad

Q#	PREGUNTA	NIVEL DE COMPLEJIDAD PARA EL LLM
Q1	¿Cuál es la temperatura actual?	Nivel básico: Esta pregunta sirve como referencia inicial. Cualquier agente LLM competente debería manejar esta tarea de recuperación de datos simple con facilidad.
Q2	¿Cuál es la intensidad de luz actual?	Nivel básico: Similar a la primera, pero evalúa si el agente puede distinguir entre diferentes tipos de datos de sensores.
Q3	¿Qué tan fuerte es la señal del sensor, considerando tanto RSSI como SNR?	Nivel intermedio: Requiere que el agente entienda y combine múltiples parámetros de comunicación, evaluando su capacidad para interpretar datos técnicos relacionados.
Q4	¿Existe riesgo de formación de niebla basado en el punto de rocío y la temperatura?	Nivel avanzado: Esta pregunta evalúa la capacidad del agente para aplicar conocimientos del dominio (meteorología) y hacer inferencias a partir de múltiples puntos de datos.
Q5	¿Cuál es el estado de salud del sensor IoT?	Nivel complejo: Requiere que el agente considere todos los datos disponibles, potencialmente infiriendo

		información faltante y realizando una evaluación holística. Esto prueba las capacidades analíticas y de razonamiento del agente.
--	--	--

16.3.2. PROCEDIMIENTO DE EJECUCIÓN DE LAS PRUEBAS

Para la ejecución de los tests se ejecutó el código desde una EC2 instance, sin embargo, para efectos de mejoramiento en la arquitectura, se puede realizar la ejecución del código desde servicios serverless de cómputo como ECS. La ejecución de las preguntas se realizó por modelo, es decir, se realizaron primero las cinco preguntas para el Modelo 1 y sucesivamente para los demás modelos. Es importante resaltar que los modelos **LLaMA-v3-70B-Instruct** y **Yi-Large** fueron utilizados a través del proveedor **fireworks.ai**.

En esta sección se ha utilizado **LangSmith** como framework de observabilidad para obtener información importante y relevante de la interacción del agente ReAct configurado con las diferentes herramientas y el uso de los modelos LLM. Por lo tanto, se usarán las siguientes métricas, descritas en el capítulo 8 del presente documento:

- **Costo Total:** Costo asociado a cada una de las llamadas realizadas al modelo por pregunta.
- **Latencia:** Tiempo total en el que se demoró el modelo en dar una respuesta a la pregunta.
- **Cantidad de Tokens Utilizados:** Número de tokens consumidos por el modelo en cada predicción.
- **Cantidad de Herramientas Utilizadas:** Número de herramientas empleadas por el agente ReAct para cada pregunta.

Adicionalmente, se desarrolló un código en Python para extraer la información de las trazas registradas en LangSmith. Mediante la configuración de la API key de LangSmith y el uso de la librería Pandas, las trazas se convirtieron en archivos CSV, facilitando su análisis posterior, para

16.4. EVALUACIÓN

Para el sistema presentado y descrito anteriormente, se plantea la selección del mejor modelo que permita obtener información tanto del sensor como de la comunicación del mismo. La evaluación del modelo se realizará en función de la calidad de los resultados, la capacidad de razonamiento, el costo y la latencia. En cuanto a la calidad de los resultados y el razonamiento, la evaluación será realizada mediante evaluación humana, enfocándose en la creatividad y relevancia de las respuestas en dicha aplicación. Para ello, se han determinado dos tipos de pruebas: una de forma binaria, utilizando únicamente un pulgar arriba si la respuesta es satisfactoria, y otra mediante una escala de calificación de 1 a 5.

16.4.1. PROCESO DE EVALUACIÓN

El proceso empleado para la evaluación se basa en combinar tanto evaluaciones cuantitativas como cualitativas, permitiendo una valoración integral del desempeño de cada modelo. A continuación, se detallan los pasos seguidos en esta etapa:

1. Diseño de Pruebas:

- Se formularon cinco preguntas (Q1 a Q5) con diferentes niveles de complejidad para evaluar las capacidades de los modelos en distintas dimensiones.
- Las preguntas abarcan desde tareas simples de recuperación de datos hasta análisis complejos que requieren inferencias y razonamiento avanzado.

2. Evaluación Humana:

- **Retroalimentación Binaria:** Los evaluadores proporcionan una retroalimentación binaria (Pulgar Arriba/Pulgar Abajo) para cada respuesta, indicando si es satisfactoria.
 - **Calificación Detallada:** Los evaluadores califican cada respuesta en una escala de 1 a 5, considerando criterios como precisión, relevancia, claridad y creatividad.
3. **Cálculo de la Puntuación Combinada:** Se utiliza una fórmula para combinar las evaluaciones binarias y detalladas, asignando un peso del 30% a la retroalimentación binaria y un 70% a las calificaciones detalladas.

Fórmula:

B = Retroalimentación binaria (1 para "Pulgar Arriba", 0 para "Pulgar Abajo"), donde la retroalimentación final binaria es la mayoría de votos.

D = Calificación detallada (promedio de calificaciones de 1 a 5 de los evaluadores)

La puntuación combinada S se calcula como:

$$S = (0.3 * B) + (0.7 * \bar{D})$$

donde:

$\bar{D}$ es el promedio de las calificaciones detalladas dadas por los evaluadores.

16.4.2. EXPLICACIÓN

Calificaciones Detalladas D :

- Los evaluadores califican la calidad de la respuesta en una escala del 1 al 5 basada en criterios específicos (precisión, relevancia, claridad, etc.).

- La calificación detallada final $\bar{D}$ es el promedio de estas calificaciones de todos los evaluadores.

Retroalimentación Binaria (B):

- Los evaluadores proporcionan una retroalimentación binaria indicando si la respuesta es generalmente buena o no (Pulgar Arriba/Pulgar Abajo).
- La retroalimentación final binaria se determina por la mayoría de los votos (por ejemplo, si la mayoría de los evaluadores dan un Pulgar Arriba, $B = 1$; de lo contrario, $B = 0$).

Ponderación:

- La retroalimentación binaria tiene un peso del 30%, elegido arbitrariamente para reflejar su importancia en la evaluación general.
- Las calificaciones detalladas tienen un peso del 70%, reflejando su evaluación exhaustiva de la calidad de la respuesta.

16.4.3. SELECCIÓN DEL "GOLDEN TEST"

Un conjunto de pruebas "Golden Test", también conocido como estándar de oro o conjunto de pruebas de referencia, es una colección predefinida de expresiones de prueba o puntos de datos que son cuidadosamente seleccionados por expertos humanos, en este caso la mejor respuesta ofrecida por los LLMs dentro del conjunto de tests realizados. Estas expresiones de prueba representan una variedad diversa de entradas y abarcan diversos escenarios para evaluar el rendimiento y la efectividad de diferentes versiones de un modelo de LLMs. El propósito de un conjunto de "Golden Test" es proporcionar una medida estandarizada y objetiva de rendimiento para diferentes iteraciones o versiones de un modelo de LLMs. Al utilizar el mismo conjunto de expresiones de prueba en múltiples

modelos, se hace posible realizar comparaciones justas y significativas entre su desempeño (sprinklr, 2024).

En este caso de estudio, se ha seleccionado la mejor respuesta obtenida de los diferentes modelos como el "Golden Test". Este enfoque permite comparar las respuestas de los modelos evaluados con una respuesta considerada óptima, garantizando una evaluación rigurosa y estandarizada, en la Tabla 3 se muestran las respuestas del Golden Test

16.4.4. RESPUESTAS DEL "GOLDEN TEST"

Tabla 3 Respuestas Golden Test, mejores respuestas dadas por los LLM, fuente: Autores

Q#	GOLDEN TEST ANSWER
Q1	The current temperature is 23.59 degrees
Q2	The current light intensity is 79.04. This value indicates a moderate level of light, which could represent conditions such as a partly cloudy day or indoor lighting, depending on the scale and units used in the measurement system.
Q3	The sensor's signal is strong, considering both RSSI and SNR values: 1. RSSI: -34.0 dBm This is a very good RSSI value, indicating a strong signal. It's well above the typical minimum of -70 dBm for reliable connections, and closer to 0 dBm (which represents an excellent signal). 2. SNR: 9.25 dB This is a good SNR value. It means the signal is 9.25 dB stronger than the noise, which is sufficient for clear communication in most scenarios. Combining these two metrics, we can conclude that the sensor has a strong and clear signal. The high RSSI suggests the sensor is likely close to the receiver or has

	a clear line of sight, while the good SNR indicates that the signal quality is high with minimal interference. This combination should result in reliable and efficient communication between the sensor and the receiving system.
Q4	Based on the current temperature of 23.59°C and dew point of 14.818°C, there is a low risk of fog formation. The temperature is significantly higher than the dew point, indicating that the air is not close to saturation, which is necessary for fog to form. However, it's important to note that weather conditions can change, and if the temperature were to drop closer to the dew point (within about 2.5°C), the risk of fog formation would increase.
Q5	The IoT sensor is healthy

16.4.5. EJEMPLO DEL PROCESO REALIZADO POR EL AGENTE REACT

A continuación se muestra la interacción y proceso realizado por el agente ReAct, en el que se evidencia las etapas de razonamiento y toma de decisiones con el uso de herramientas la siguiente interacción se ha traducido para comprensión del lector, sin embargo el total de las respuestas originales se encuentra en el apéndice del presente documento. En la Figura 20 se muestra el proceso realizado por el agente ReAct.

Humano: ¿Cuál es el estado de salud del sensor IoT?

Razonamiento Agente: Para determinar el estado de salud del sensor IoT, debemos verificar algunas variables clave de comunicación que indiquen la calidad de la señal y la conectividad. Las variables más relevantes para esto son RSSI (Received Signal Strength Indicator) y SNR (Signal-to-Noise Ratio). Vamos a recuperar estos valores.

Mensaje de la Herramienta:

Acción: get_communication

Entrada: rssi

Resultado: -34.0

Razonamiento Agente: Hemos recibido el valor de RSSI, que es -34.0 dBm. Este es un valor de señal bastante bueno, ya que los valores de RSSI típicamente oscilan entre -100 dBm (muy débil) y 0 dBm (excelente). Un valor de -34.0 dBm indica una señal fuerte. Sin embargo, vamos a verificar también el SNR para obtener una imagen más completa de la salud de la comunicación del sensor.

Mensaje de la Herramienta:

Acción: get_communication

Entrada: snr

Resultado: 9.25

Razonamiento Agente: Hemos recibido el valor de SNR, que es 9.25 dB. El SNR es una medida de la fuerza de la señal en relación con el ruido de fondo, y un valor más alto indica una señal más clara. Un SNR de 9.25 dB se considera bueno para aplicaciones IoT.

Ahora que tenemos ambos valores, RSSI y SNR, podemos evaluar la salud general del sensor IoT:

RSSI: -34.0 dBm (Señal fuerte)

SNR: 9.25 dB (Buena relación señal-ruido)

Basándonos en estos valores, podemos concluir que el sensor IoT está en buen estado. El RSSI fuerte indica que el sensor tiene una conexión robusta a la red, y el buen SNR sugiere que la calidad de la señal es clara con mínima interferencia.

Acción del Agente Final:

Action: AgentFinish

Salida del Agente: El sensor IoT parece estar en buen estado. Tiene una señal fuerte (RSSI de -34.0 dBm) y una buena relación señal-ruido (SNR de 9.25 dB). Estos valores indican que el sensor tiene una conexión robusta a la red y está transmitiendo datos con mínima interferencia. La calidad de la comunicación es alta, lo que sugiere que el sensor está funcionando bien y enviando datos de manera fiable.

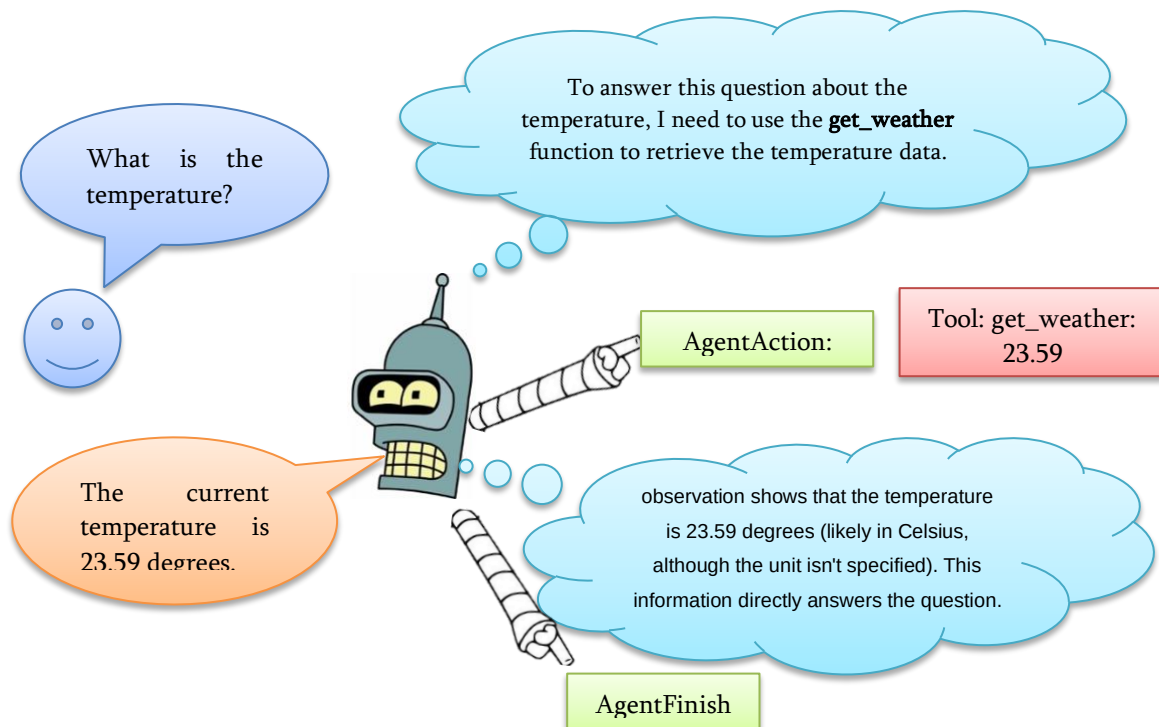


Figura 20 Proceso de razonamiento y actuación del agente, pregunta Q1. Fuente Autores

16.4.6. OBTENCIÓN DE RESULTADOS

Para obtener el valor final de los criterios de calidad de razonamiento y resultados, se promediaron las calificaciones de 1 a 5 de cada evaluador. En cuanto a los resultados binarios, se utilizó la mayoría de los votos como el resultado final de esta evaluación. Finalmente, el resultado total de razonamiento se calculó aplicando un peso del 30% a la retroalimentación binaria y un 70% a los resultados promedios de la evaluación detallada, conforme a la fórmula presentada anteriormente.

Este enfoque permitió una evaluación equilibrada, considerando tanto la percepción general de los evaluadores mediante la retroalimentación binaria como una evaluación más detallada y cuantitativa a través de las calificaciones en escala.

Comparación con el "Golden Test"

Todas las respuestas proporcionadas por los diferentes modelos fueron comparadas con el "Golden Test" para determinar su exactitud y coherencia. El modelo que más se acercó a la respuesta ideal en términos de precisión, relevancia y claridad fue seleccionado como el mejor modelo para la aplicación IoT específica.

Tabla de Comparación de Resultados

A continuación, se muestra la comparación de los resultados a la pregunta, sobre la intensidad lumínica actual, Tabla 4.

Tabla 4 Métricas cuantitativas y cualitativas de los diferentes LLMs, Pregunta Q2. Fuente: Autores

WHAT IS THE CURRENT LIGHT INTENSITY?						
MODELO	VERSIÓN (Junio/24)	TOKENS	LATENCIA	COSTO (USD)	CALIDAD RAZONAMIENTO	CALIDAD RESULTADO
01.AI	yi-large	943	00:02,587	\$0,002829	3,1	3,8
ANTHROPIC	claude-3-5-sonnet-20240620	983	00:06,186	\$0,002949	3,8	3,8
OPENAI	gpt-4o	756	00:03,846	\$0,003780	3,1	3,8
META AI	llama-v3-70b-instruct	771	00:02,668	\$0,000732	3,1	3,8

16.5. ANÁLISIS

Una vez obtenidos los resultados de la evaluación humana y las métricas registradas por LangSmith, se procede a analizar el desempeño de los diferentes modelos de lenguaje en

función de las variables definidas durante la etapa de identificación. Este análisis se realiza considerando la calidad de los resultados, la capacidad de razonamiento, el costo y la latencia de cada modelo en relación con las preguntas formuladas (Q1 a Q5) de diferentes niveles de complejidad.

16.5.1. PROCESO DE ANÁLISIS

Para comparar de manera efectiva los resultados obtenidos, se utilizaron gráficas de radar las cuales permiten visualizar y contrastar múltiples variables de desempeño en un solo gráfico. Este tipo de gráfica es especialmente útil para representar diferentes niveles de complejidad de las preguntas y evaluar cómo cada modelo se desempeña en cada una de ellas.

16.5.2. NORMALIZACIÓN DE VALORES

Antes de la representación gráfica, es necesario normalizar los valores obtenidos para cada variable de evaluación. La normalización facilita la comparación al escalar los valores entre 0 y 1. La fórmula utilizada para la normalización es la siguiente:

$$N_{ij} = \frac{V_{ij}}{V_{j,max}}$$

donde:

- V_{ij} = Valor de la variable j para la pregunta i .
- $V_{j,max}$ = Valor máximo de la variable j entre todas las preguntas.
- N_{ij} = Valor normalizado de la variable j para la pregunta i .

Este procedimiento asegura que todas las variables sean comparables en una escala uniforme, permitiendo una interpretación más clara de los resultados.

16.5.3. RESULTADOS POR PREGUNTA

A continuación, se presentan los análisis específicos para cada pregunta, destacando el desempeño de los modelos evaluados. Para obtener estas gráficas, se normalizaron los valores de cada variable dividiendo el valor actual por el valor máximo de dicha variable entre todas las preguntas. Este proceso permite una comparación sencilla y efectiva de las capacidades de cada modelo en distintos aspectos evaluados.

Pregunta Q1: ¿Cuál es la temperatura actual?

Para la pregunta Q1, Figura 21, se observó que los modelos presentaron una calidad de razonamiento buena, aunque el modelo LLaMA-v3-70B-Instruct recibió una menor valoración en la evaluación humana debido a que, en ocasiones, solo proporcionó el valor sin detalles adicionales. Sin embargo, este modelo destacó por tener uno de los costos más económicos en el mercado. En contraste, otros modelos como GPT4o y Claude-3-5-sonnet ofrecieron respuestas más completas pero a un costo significativamente mayor.

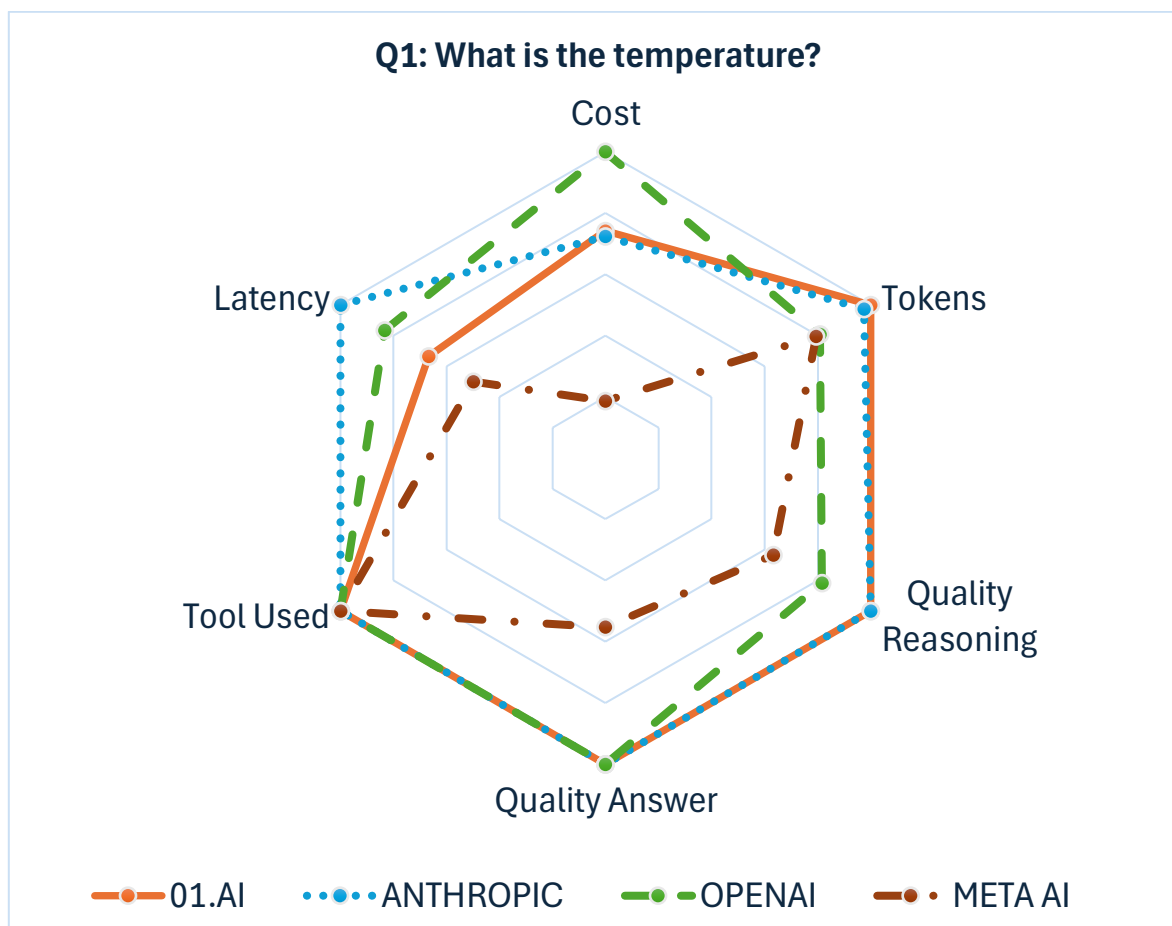


Figura 21 Gráfica de radar para la pregunta Q1. Fuente: Autores

Pregunta Q2: ¿Cuál es la intensidad de luz actual?

En la pregunta Q2, Figura 22, los modelos mostraron una mejor respuesta en comparación con Q1, evidenciando una mayor capacidad para proporcionar información detallada y relevante. Esto indica una mejora en la habilidad de los modelos para distinguir y describir diferentes tipos de datos de sensores.

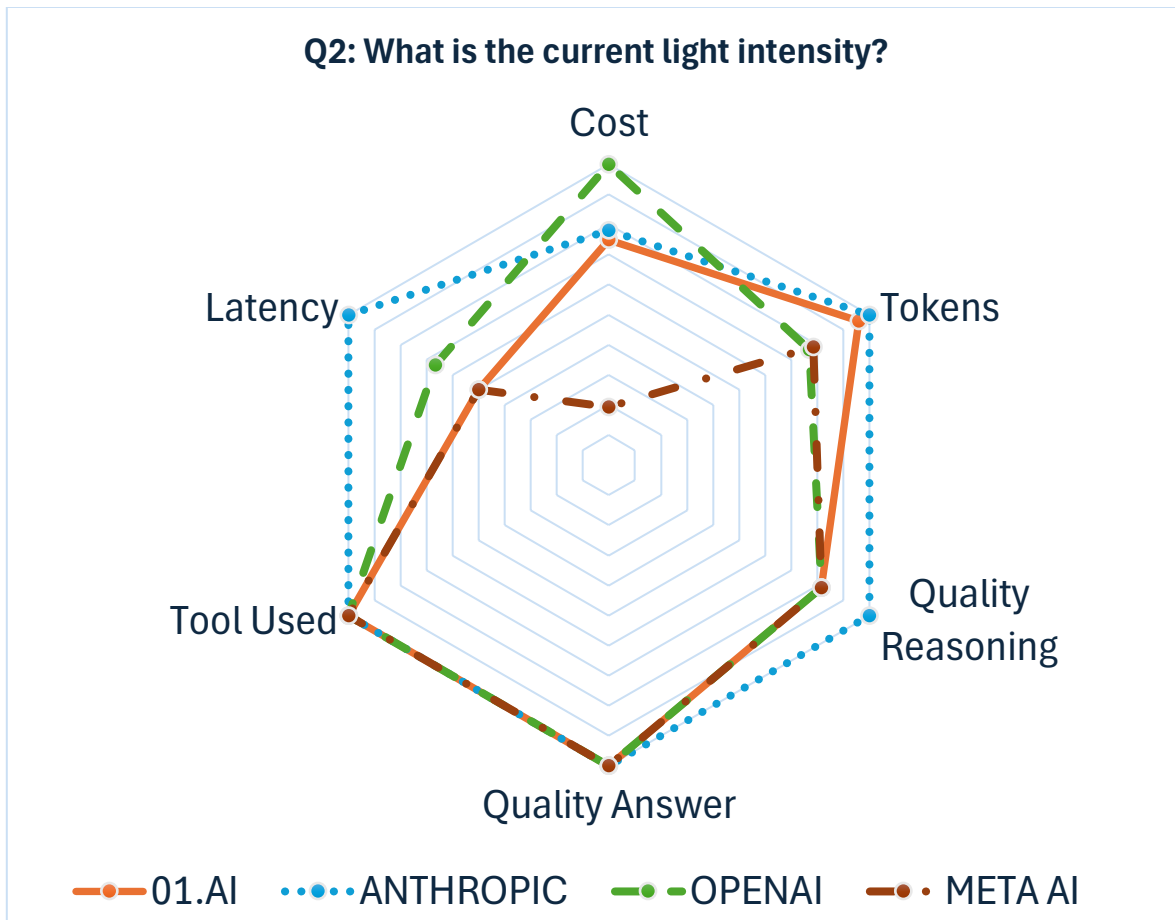


Figura 22 Gráfica de radar para la pregunta Q2. Fuente: Autores

Pregunta Q3: ¿Qué tan fuerte es la señal del sensor, considerando tanto RSSI como SNR?

Para las preguntas Q3 y Q4, se observó que los modelos Claude-3-5-sonnet y LLaMA-v3-70B-Instruct utilizó una mayor cantidad de tokens y herramientas y herramientas, lo que se tradujo en una latencia más alta. Además, se detectaron problemas en la invocación correcta de las funciones necesarias para procesar estas preguntas, lo que incrementó el uso de tokens. No obstante, se destaca que LLaMA-v3-70B-Instruct sigue siendo una opción costo-efectiva en el mercado, como se muestra en la Figura 23.

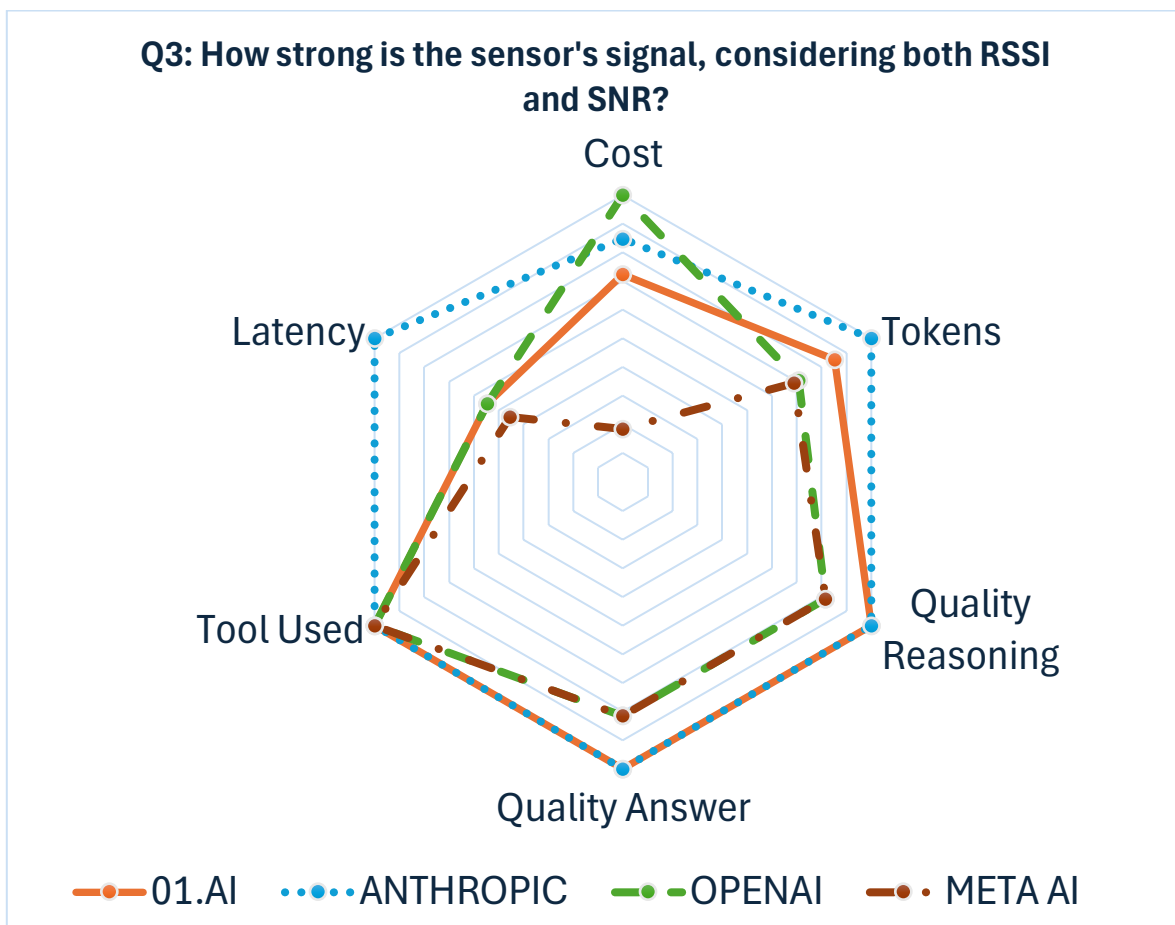


Figura 23 Gráfica de radar para la pregunta Q3. Fuente: Autores

Pregunta Q4: ¿Existe riesgo de formación de niebla basado en el punto de rocío y la temperatura?

Como se observa en la Figura 24, En la pregunta Q4, el desempeño de los modelos fue consistente con la evaluación de Q3, mostrando una adecuada capacidad para analizar múltiples variables y realizar inferencias basadas en conocimientos del dominio (meteorología). Sin embargo, **LLaMA** continuó presentando un mayor uso de recursos, aunque su costo reducido lo hace atractivo para aplicaciones con presupuesto limitado.

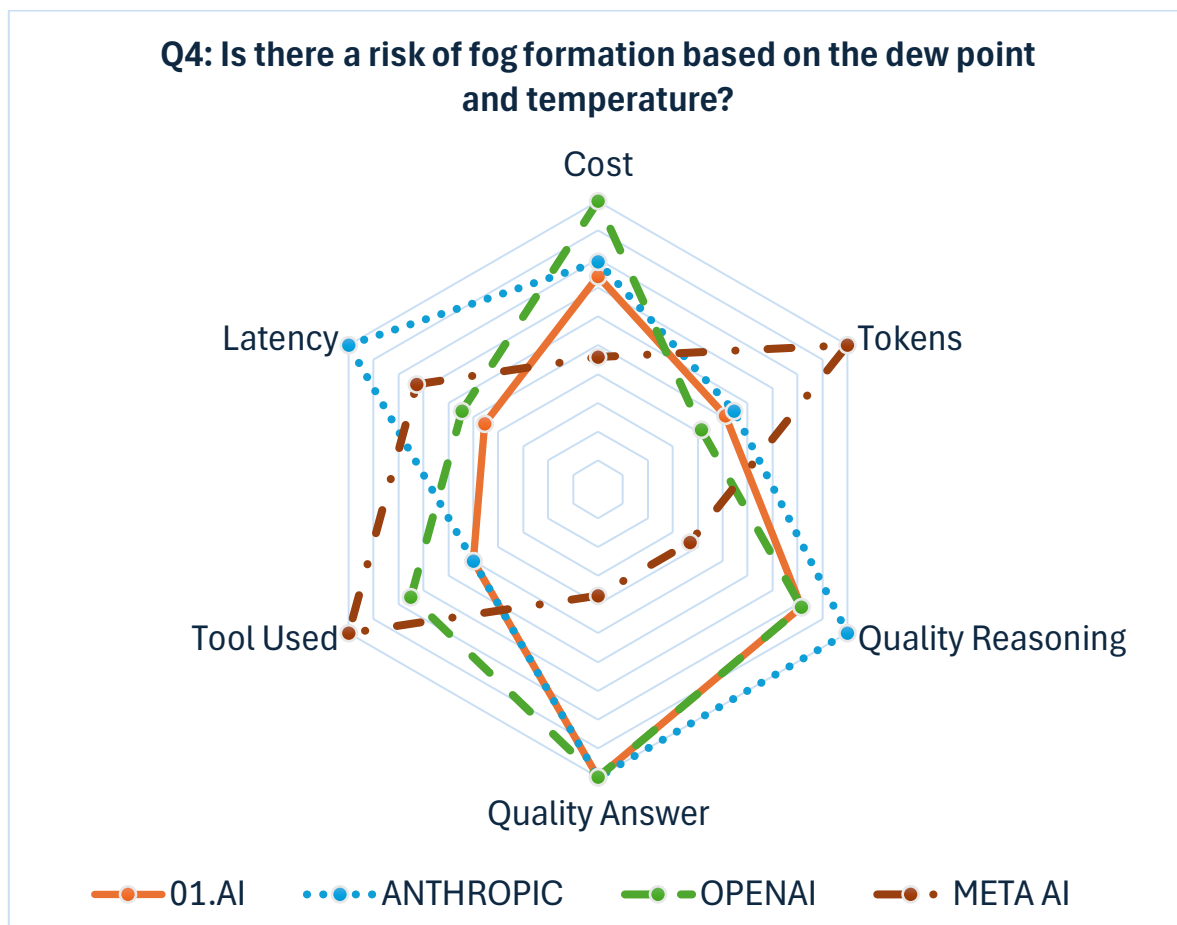


Figura 24 Gráfica de radar para la pregunta Q4. Fuente: Autores

Pregunta Q5: ¿Cuál es el estado de salud del sensor IoT?

Finalmente, para la pregunta Q5, Figura 25, una de las más complejas en términos generales, los modelos demostraron una alta calidad en las respuestas. GPT4o y Claude-3-5-sonnet sobresalió en precisión y claridad, aunque su costo total de GPT4o es elevado en comparación con otros modelos. Los modelos evaluados lograron proporcionar evaluaciones holísticas del estado del sensor, lo que confirma su capacidad para manejar consultas complejas de manera eficiente.

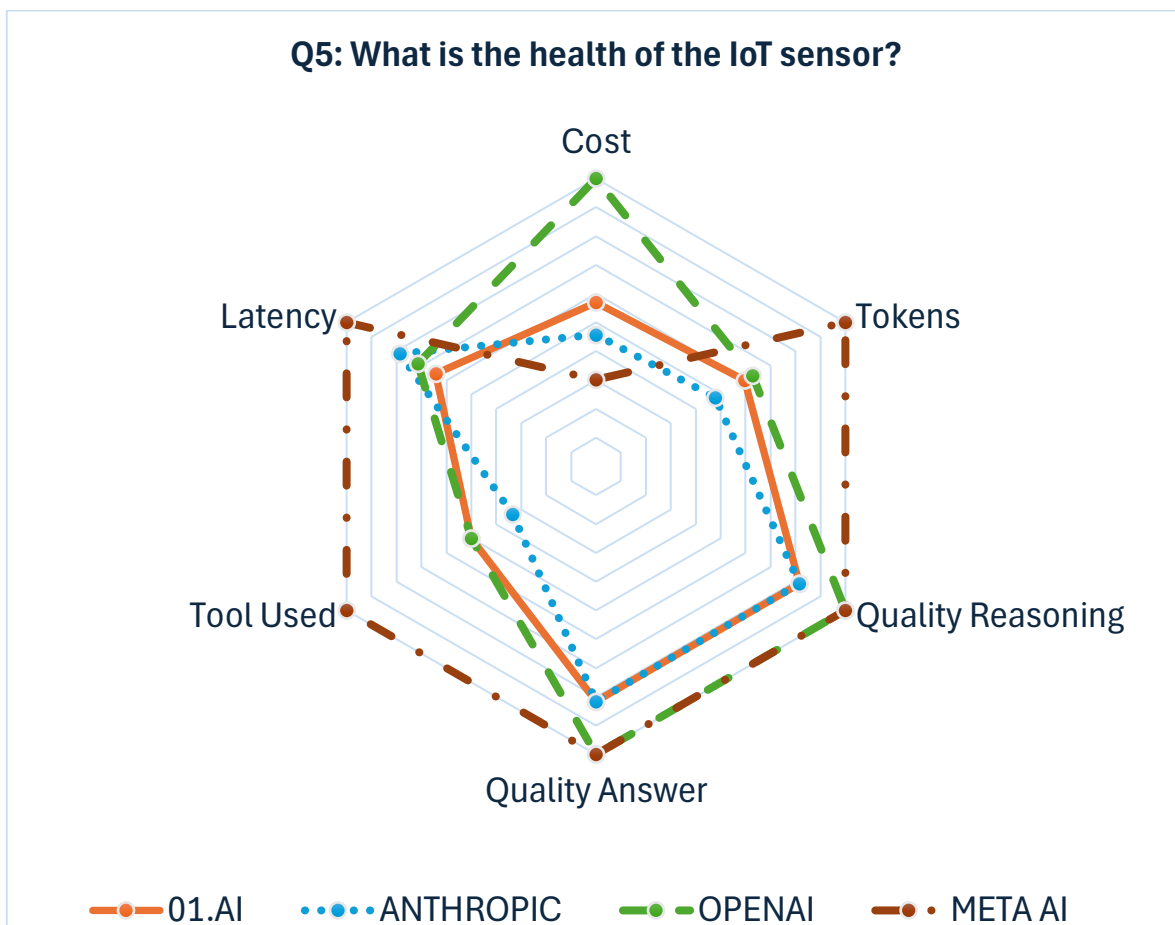


Figura 25 Gráfica de radar para la pregunta Q5. Fuente: Autores

Comparación General de Modelos

Al sumar las características evaluadas para cada pregunta, se obtiene una visión integral del desempeño de los modelos. La Figura 26 siguiente ilustra una comparación general basada en las puntuaciones combinadas obtenidas en cada pregunta.

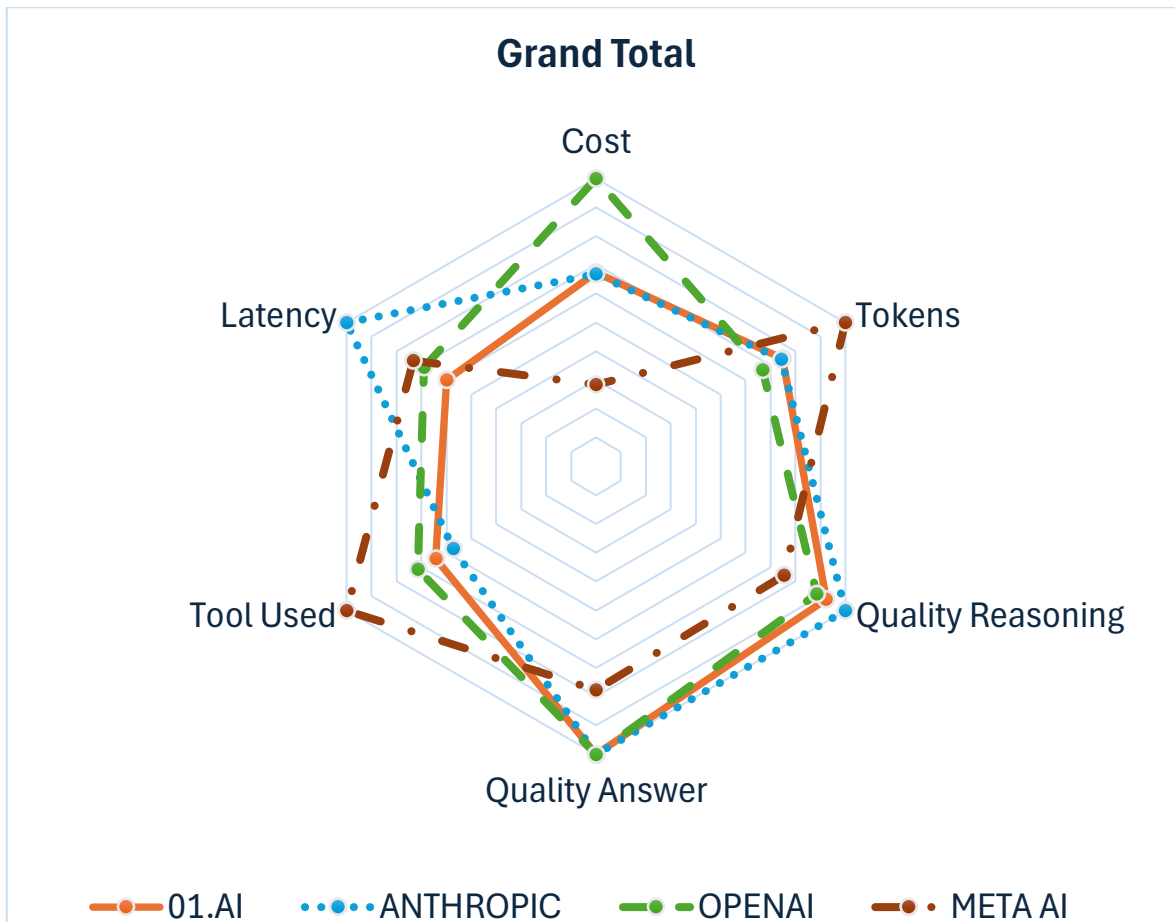


Figura 26 Gráfica de radar resultado de Evaluación para todos los LLMs. Fuente: Autores

El modelo OPENAI GPT4-o es uno de los más costosos en comparación con los modelos evaluados. Sin embargo, ofrece respuestas de calidad y precisión, similar al modelo Claude-3-5-sonnet, lo que lo hace adecuado para aplicaciones donde la exactitud y la claridad son prioritarias. Por otro lado, modelos como LLaMA y Yi-Large presentan un equilibrio entre costo y desempeño, siendo opciones viables para implementaciones con restricciones presupuestarias.

17. ANÁLISIS DE RESULTADOS DE LA APLICACIÓN DE LA METODOLOGÍA AL CASO DE ESTUDIO

La presente investigación ha permitido evaluar y seleccionar LLM adecuado para la aplicación de monitoreo climático en un entorno de home office, considerando tanto la calidad de las respuestas como la eficiencia operativa. A través de una metodología rigurosa que combina evaluaciones humanas con métricas cuantitativas, se ha logrado identificar el modelo que mejor se adapta a las necesidades específicas del sistema IoT implementado.

El análisis de resultados obtenido a partir de la evaluación comparativa de los LLMs — Claude-3-5-Sonnet (Anthropic), GPT-4o (OpenAI), LLaMA-v3-70B (Meta AI) y Yi-Large (01.AI)— proporciona una visión integral sobre su desempeño en el contexto de un sistema IoT para monitoreo meteorológico en un entorno de home office. A continuación, se detallan los principales diferencias y su interpretación en relación con los objetivos planteados.

Costo de Implementación: Claude-3-5-Sonnet y LLaMA-v3-70B demostraron una mayor eficiencia en costos operativos, lo cual los posiciona como opciones viables para implementaciones a gran escala donde el control de gastos es prioritario. Esta eficiencia puede atribuirse a su menor consumo de tokens y optimización en el uso de recursos, alineándose con estudios previos que sugieren que la optimización de recursos es crucial para aplicaciones IoT con limitaciones de presupuesto.

GPT-4o, aunque presenta una calidad de respuesta y capacidad de razonamiento, incurre en costos operativos elevados debido a su valor de tokens. Este factor puede limitar su adopción en entornos donde el presupuesto es una restricción significativa.

LLaMA-v3-70B muestra un costo moderado, ofreciendo un equilibrio entre rendimiento y gasto, lo que lo hace adecuado para aplicaciones que requieren precisión sin comprometer excesivamente el presupuesto.

Latencia: Yi-Large se destacó por sus bajos tiempos de respuesta, un aspecto crítico en aplicaciones IoT que demandan interacciones en tiempo real. La baja latencia facilita una

respuesta rápida y eficiente, mejorando la experiencia del usuario y la efectividad del sistema.

GPT-4o y LLaMA-v3-70B presentaron latencias moderadas, lo que puede ser aceptable dependiendo de los requerimientos específicos de la aplicación. Sin embargo, en escenarios donde la rapidez es esencial, estos modelos podrían presentar desventajas.

Calidad de la Respuesta: GPT-4o y Claude-3-5-Sonnet sobresalieron en términos de precisión y coherencia, especialmente en escenarios de alta complejidad. Su capacidad para generar respuestas detalladas y contextualmente relevantes lo hace ideal para aplicaciones donde la exactitud es crucial.

Yi-Large en ocasiones careció de detalles contextuales, lo que puede afectar la interpretación de los datos en situaciones complejas.

LLaMA-v3-70B proporcionó respuestas adecuadas pero con menos detalle en escenarios de mayor complejidad, lo que sugiere que, aunque es eficiente, podría requerir ajustes adicionales para mejorar su desempeño en tareas más sofisticadas.

Capacidad de Razonamiento: Claude-3-5-Sonnet y GPT-4o demostraron una excelente capacidad de razonamiento, siendo capaces de realizar inferencias complejas y mantener coherencia lógica en sus respuestas. Esta capacidad es fundamental para aplicaciones que requieren un análisis profundo de múltiples variables ambientales.

LLaMA-v3-70B y Yi-Large realizaron inferencias precisas, aunque su rendimiento fue menos consistente en desafíos más complejos. Esto sugiere que, si bien son capaces de manejar tareas básicas y algunas intermedias, podrían enfrentarse a dificultades en situaciones que demandan un razonamiento más avanzado.

Uso de Tokens y Recursos: LLaMA-v3-70B y GPT-4o utilizaron una mayor cantidad de tokens y recursos, lo que impacta directamente en los costos y la eficiencia operativa del sistema. Este alto consumo puede ser una barrera para su implementación en entornos con recursos limitados.

Yi-Large fue el más eficiente en términos de uso de recursos, lo que lo hace particularmente beneficioso para entornos donde la optimización es esencial. Este aspecto se alinea con la

necesidad de minimizar el consumo de recursos en aplicaciones IoT para maximizar la sostenibilidad y reducir costos a largo plazo. En la Tabla 5 se muestra la comparación de los LLMs de acuerdo con las métricas seleccionadas.

Comparación de Modelos

Tabla 5 Comparación de LLMs de acuerdo con las métricas seleccionadas. Fuente: Autores

Criterio	Claude-3-5-Sonnet (Anthropic)	GPT-4o (OpenAI)	LLaMA-v3-70B (Meta AI)	Yi-Large (01.AI)
Costo de Implementación	Eficiente en costos operativos	Alto costo por complejidad	Moderado	Eficiente
Latencia	Baja	Moderada	Moderada	Ligeramente alta
Calidad de la Respuesta	Adecuada, menor detalle en alta complejidad	Alta precisión, coherente	Buena, contexto limitado	Adecuada, menos detalle
Capacidad de Razonamiento	Excelente	Excelente	Buena, inconsistente en desafíos	Adecuada
Uso de Tokens y Recursos	Moderado	Alto	Alto, más tokens usados	Bajo, eficiente

La comparación de los cuatro modelos permite concluir que cada uno presenta fortalezas y debilidades que los hacen adecuados para diferentes escenarios de aplicación dentro de un sistema de monitoreo IoT. Es importante destacar que la elección del modelo ideal depende de los requerimientos específicos del entorno de implementación, considerando las restricciones presupuestarias, la calidad requerida de las respuestas y la capacidad de manejar interacciones complejas.

No necesariamente por el uso de mayor cantidad de tokens o de herramientas signifique que sean buen modelo para aplicaciones de IoT sin embargo los resultados obtenidos con los mismos que con otros modelos. Para evaluar las respuestas entregadas por los agentes se sugiere que la evaluación humana no conozca el origen del modelo para evitar selección y elección en particular. La mayoría de las respuestas usadas como Golden test fueron por parte del modelo de Anthropic (version junio 2024).

10. CONCLUSIONES

La metodología desarrollada se fundamenta en un proceso estructurado que incluye las fases de Identificar, Desarrollar, Testear y Monitorear, Evaluar y Analizar, abarcando todas las etapas críticas para la selección de Modelos de Lenguaje de Gran Tamaño. Desde la recolección y preprocesamiento de datos hasta la evaluación exhaustiva de los modelos, cada fase ha sido diseñada para asegurar una implementación rigurosa y eficiente. La inclusión del agente ReAct resultó fundamental para optimizar la toma de decisiones en el uso de herramientas, demostrando un rendimiento elevado al integrar capacidades de razonamiento dentro del sistema IoT. Además, la incorporación de casos de estudio reales permitió evaluar la efectividad de los LLMs en escenarios controlados, asegurando su adecuación para situaciones reales de monitoreo climático. Este enfoque iterativo y cíclico facilita la continua optimización de la selección de LLMs y sus componentes asociados, garantizando la fiabilidad y eficiencia del sistema a lo largo del tiempo.

El modelo de predicción de temperatura utilizado en el caso de estudio presentó limitaciones significativas en su rendimiento, evidenciadas por métricas bajas como un valor de R^2 de 0.0039, lo que indica una incapacidad para explicar la variabilidad de los datos de temperatura. La alta variabilidad observada en los resultados de la validación cruzada, con un error cuadrático medio (MSE) de 1.4458 en un fold, sugiere una inconsistencia en la capacidad del modelo para predecir eficazmente los patrones climáticos. No obstante, el uso del agente ReAct demostró ser crucial para mejorar la precisión y exactitud del sistema, al detectar anomalías en los datos y recomendar el uso de fuentes externas más confiables, lo que permitió mantener una alta fiabilidad en el funcionamiento del sistema IoT.

Un logro destacado de esta investigación fue la identificación y selección de un conjunto de métricas cuantitativas y cualitativas adecuadas para evaluar la precisión y exactitud de los LLMs en comparación con los modelos convencionales. Las métricas utilizadas incluyen la calidad de las respuestas, capacidad de razonamiento, costo, latencia, y uso de tokens y recursos. La combinación de métricas automáticas con evaluaciones humanas expertas proporciona una visión integral del rendimiento de cada modelo, facilitando una comparación clara y fundamentada entre tecnologías nuevas y tradicionales. Además, se desarrolló una métrica específica para la probabilidad de selección de herramientas por

parte del agente, destacando la predominancia de la obtención de temperatura con una probabilidad del 30%, lo que refleja la frecuencia y relevancia de esta herramienta en el sistema.

El desarrollo de una plataforma en la nube fue esencial para gestionar la gran cantidad de datos generados por los dispositivos IoT y para permitir la integración eficiente de los LLMs. La arquitectura serverless, basada en tecnologías como AWS IoT Core, Amazon Timestream y AWS Lambda Functions, se diseñó para ser escalable y facilitar el acceso eficiente a datos en tiempo real. Esta plataforma garantizó que los LLMs pudieran procesar información meteorológica con rapidez y precisión, cumpliendo con los requerimientos de disponibilidad para tareas críticas de predicción. Además, la plataforma permitió el monitoreo y evaluación continua de los modelos, creando un entorno integrado donde cada componente colabora de manera efectiva para el procesamiento de datos, lo que sienta una base sólida para futuras aplicaciones y mejoras en el ámbito del Internet de las Cosas y la inteligencia artificial aplicada al monitoreo climático.

11. REFERENCIAS

- AWS. (2024a). *Amazon Timestream*. AWS. <https://docs.aws.amazon.com/timestream/latest/developerguide/what-is-timestream.html>
- AWS. (2024b). *AWS IoT Core*. AWS. <https://aws.amazon.com/iot-core/>
- AWS. (2024c). *AWS Lambda*. AWS. <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>
- Breiman, L. (2001). *Random Forests* (Vol. 45). <https://doi.org/https://doi.org/10.1023/A:1010933404324>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodai, D. (2020). *Language Models are Few-Shot Learners*. <http://arxiv.org/abs/2005.14165>
- Dai, X., Liu, G. P., Hu, W., Lei, Z., & Zhou, H. (2023). Learning from ChatGPT: A Transformer-Based Model for Wind Power Forecasting. *Proceedings - 2023 IEEE International Conference on Environment and Electrical Engineering and 2023 IEEE Industrial and Commercial Power Systems Europe, IEEEIC / I and CPS Europe 2023*. <https://doi.org/10.1109/IEEEIC/ICPSEurope57605.2023.10194807>
- Grafana. (2024). *Grafana Labs*. <https://grafana.com/docs/>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). Springer Series in Statistics The Elements of Statistical Learning - Data Mining, Inference, and Prediction. In *Springer* (Vol. 2nd).
- langchain. (2024). *ReAct implementation*. https://langchain-ai.github.io/langgraph/concepts/agentic_concepts/#react-implementation
- Langsmith. (2024). *Concepts* - *Langsmith*. <https://docs.smith.langchain.com/evaluation/concepts>

- Lee, Y. Y., Kim, W. M., Sohn, S. J., Kim, B. R., & Seuseu, S. K. (2022). Advances and challenges of operational seasonal prediction in Pacific Island Countries. *Scientific Reports*, 12(1). <https://doi.org/10.1038/s41598-022-15345-w>
- Liao, Y., Loures, E. D. F. R., & Deschamps, F. (2018). Industrial Internet of Things: A Systematic Literature Review and Insights. *IEEE Internet of Things Journal*, 5(6), 4515–4525. <https://doi.org/10.1109/JIOT.2018.2834151>
- Lin, J., Yu, W., Zhang, N., Yang, X., Zhang, H., & Zhao, W. (2017). A Survey on Internet of Things: Architecture, Enabling Technologies, Security and Privacy, and Applications. *IEEE Internet of Things Journal*, 4, 1125–1142. <https://api.semanticscholar.org/CorpusID:31245252>
- Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest. *Proceedings - IEEE International Conference on Data Mining, ICDM*. <https://doi.org/10.1109/ICDM.2008.17>
- Mekki, K., Bajic, E., Chaxel, F., & Meyer, F. (2019). A comparative study of LPWAN technologies for large-scale IoT deployment. *ICT Express*, 5(1), 1–7. <https://doi.org/10.1016/j.icte.2017.12.005>
- Mell, P. M., & Grance, T. (2011). *The NIST definition of cloud computing*. <https://doi.org/10.6028/NIST.SP.800-145>
- Nascimento, N., Alencar, P., & Cowan, D. (2023a). GPT-in-the-Loop: Supporting Adaptation in Multiagent Systems. *Proceedings - 2023 IEEE International Conference on Big Data, BigData 2023*, 4674–4683. <https://doi.org/10.1109/BigData59044.2023.10386490>
- Nascimento, N., Alencar, P., & Cowan, D. (2023b). GPT-in-the-Loop: Supporting Adaptation in Multiagent Systems. *Proceedings - 2023 IEEE International Conference on Big Data, BigData 2023*, 4674–4683. <https://doi.org/10.1109/BigData59044.2023.10386490>
- Pedregosa FABIANPEDREGOSA, F., Michel, V., Grisel OLIVIERGRISEL, O., Blondel, M., Prettenhofer, P., Weiss, R., Vanderplas, J., Cournapeau, D., Pedregosa, F., Varoquaux, G., Gramfort, A., Thirion, B., Grisel, O., Dubourg, V., Passos, A., Brucher, M., Perrot and Édouardand, M., Duchesnay, and Édouard, & Duchesnay EDOUARD DUCHESNAY, Fré. (2011). Scikit-learn: Machine Learning in Python Gaël

Varoquaux Bertrand Thirion Vincent Dubourg Alexandre Passos PEDREGOSA, VAROQUAUX, GRAMFORT ET AL. Matthieu Perrot. In *Journal of Machine Learning Research* (Vol. 12). <http://scikit-learn.sourceforge.net>.

Petrovic, N., Konicanin, S., & Suljovic, S. (2023a). ChatGPT in IoT Systems: Arduino Case Studies. *2023 IEEE 33rd International Conference on Microelectronics, MIEL 2023*. <https://doi.org/10.1109/MIEL58498.2023.10315791>

Petrovic, N., Konicanin, S., & Suljovic, S. (2023b). ChatGPT in IoT Systems: Arduino Case Studies. *2023 IEEE 33rd International Conference on Microelectronics, MIEL 2023*. <https://doi.org/10.1109/MIEL58498.2023.10315791>

PremSankar, G., Di Francesco, M., & Taleb, T. (2018). Edge Computing for the Internet of Things: A Case Study. *IEEE Internet of Things Journal*, 5(2), 1275–1284. <https://doi.org/10.1109/JIOT.2018.2805263>

Satyanarayanan, M. (n.d.). *The Emergence of Edge Computing*.

Sornin, N., Luis, M., Eirich, T., Kramp, T., & Hersent, O. (2015). *LoRaWAN Specification. LoRa Alliance*.

sprinklr. (2024). *What is a Golden Test Set*. <https://www.sprinklr.com/help/articles/set-up-nlu-based-intents/what-is-a-golden-test-set/64804933723d925979db84e4>

Sri Preethaa, K. R., Muthuramalingam, A., Natarajan, Y., Wadhwa, G., & Ali, A. A. Y. (2023). A Comprehensive Review on Machine Learning Techniques for Forecasting Wind Flow Pattern. In *Sustainability (Switzerland)* (Vol. 15, Issue 17). Multidisciplinary Digital Publishing Institute (MDPI). <https://doi.org/10.3390/su151712914>

symflower. (2024). *Why monitor LLMs*. <https://symflower.com/en/company/blog/2024/llm-observability/>

Tokognon, A. C., Gao, B., Tian, G. Y., & Yan, Y. (2017). Structural Health Monitoring Framework Based on Internet of Things: A Survey. *IEEE Internet of Things Journal*, 4(3), 619–635. <https://doi.org/10.1109/JIOT.2017.2664072>

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. <http://arxiv.org/abs/1706.03762>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. <http://arxiv.org/abs/2210.03629>
- Yu, W., Liang, F., He, X., Hatcher, W. G., Lu, C., Lin, J., & Yang, X. (2017). A Survey on the Edge Computing for the Internet of Things. In *IEEE Access* (Vol. 6, pp. 6900–6919). Institute of Electrical and Electronics Engineers Inc. <https://doi.org/10.1109/ACCESS.2017.2778504>
- Zanella, A., Bui, N., Castellani, A., Vangelista, L., & Zorzi, M. (2014). Internet of things for smart cities. *IEEE Internet of Things Journal*, 1(1), 22–32. <https://doi.org/10.1109/JIOT.2014.2306328>
- Zhang, J., Liu, P., Zhang, F., Iwabuchi, H., De Moura, A. A. D. H. E. A., & De Albuquerque, V. H. C. (2021). Ensemble Meteorological Cloud Classification Meets Internet of Dependable and Controllable Things. *IEEE Internet of Things Journal*, 8(5), 3323–3330. <https://doi.org/10.1109/JIOT.2020.3043289>
- Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). *Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting*. www.aiai.org

12. APÉNDICE

12.0. CONFIGURACIÓN DEL GATEWAY RAK EN AWS

Esta sección tiene como objetivo detallar los pasos necesarios para configurar un gateway RAK7268 para conectarse a AWS IoT Core, siguiendo un enfoque claro y detallado. Se recomienda contar con experiencia previa en configuración de redes y uso de plataformas IoT para facilitar la comprensión del proceso.

1. Configuración del Hardware del Gateway

Para iniciar la configuración del Gateway RAK7268, es necesario referirse a la documentación oficial del producto. En esta, se pueden encontrar descripciones detalladas del hardware y de sus características específicas. Asegúrese de seguir todas las recomendaciones de instalación física del dispositivo.

2. Configuración del Software del Gateway

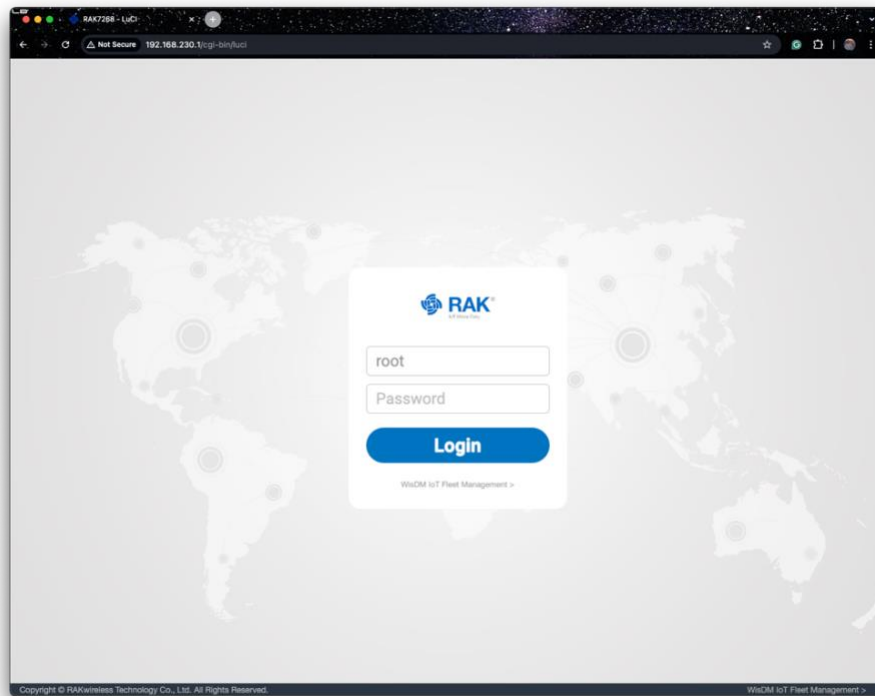


Figura 27 Pantalla de Inicio Gateway RAK. Fuente: Autores

Para configurar el software del Gateway RAK7268, siga los pasos descritos a continuación:

1. Por defecto, el Gateway opera en el modo "Wi-Fi AP". Esto significa que se puede localizar un SSID con el nombre "RAK7268_XXXX" en la lista de redes Wi-Fi de su computadora. "XXXX" corresponde a los últimos dos bytes de la dirección MAC del Gateway.
2. Para acceder a la plataforma de gestión web del dispositivo, ingrese la dirección IP 192.168.230.1 en su navegador.
3. Utilice las siguientes credenciales para iniciar sesión:
 - **Usuario:** root
 - **Contraseña:** root

La versión de firmware que soporta AWS IoT Core para LoRaWAN es la 1.2.0065_Release_r209. Para actualizar el firmware, navegue a **System -> Backup/Flash Firmware -> Flash new firmware image** y actualice la imagen del firmware.

4. configuración del Modo de Red a "Basic Station"

The screenshot shows the RAK LoRaWAN Network Settings interface. The left sidebar contains navigation links: Status, Network, Channel Plan, LoRa Network, Network Settings, Services, System, and WisDM. The main content area is titled 'LoRaWAN Network Settings' and includes a 'Gateway EUI' field with the value 'act109fffe05fe30'. Below this, the 'Mode' is set to 'Basic Station' and the 'Log Level' is set to 'DEBUG'. The 'LoRa Basic Station' section is titled 'Basic Station Configuration'. It includes a 'Server' selection with radio buttons for 'CUPS Boot Server', 'CUPS Server', and 'LNS Server' (which is selected). The 'URI' field contains 'wss://A2WU4YJLCH5P37.lns.lorawan.us-east-1.amazonaws.com'. The 'Port' is set to '443'. The 'Authentication Mode' is set to 'TLS Server and Client Authentication'. Below these fields are three text areas for 'trust', 'certificate', and 'key', each containing a long base64-encoded string. At the bottom right, there are 'Save & Apply' and 'Reset' buttons. The footer of the interface shows 'Copyright © RAKwireless Technology Co., Ltd. All Rights Reserved.' and 'WisDM IoT Fleet Management >'.

Figura 28 Gateway RAK, interfaz de configuración "basic station". Fuente: Autores

1. Diríjase a **LoRa Network -> Network Settings**.
2. Cambie el modo en la configuración de la red LoRaWAN a **Basic Station**.
3. Seleccione **LNS Server** en el campo de **Server** y luego **TLS Server and Client Authentication** en **Authentication Mode**.

Esta configuración permite que el Gateway se conecte al servicio AWS IoT Core para LoRaWAN, el cual ofrece una plataforma centralizada para gestionar dispositivos IoT.

Configuración del URI, Puerto y Modo de Autenticación

Una vez configurado el modo de red, ingrese la información del URI, el puerto y el modo de autenticación según los detalles de AWS IoT Core. Esto asegurará que la conexión entre el Gateway y el servidor LNS de AWS sea exitosa.

4. Verificar la Conexión del Gateway

Para verificar la conexión del Gateway con AWS IoT Core, siga las instrucciones del manual en línea para verificar el estado de conexión del dispositivo mediante la consola de AWS. Asegúrese de que el Gateway se encuentre correctamente conectado y operativo.

6. Agregar un Dispositivo Final (End Device)

1. Para habilitar la comunicación con el Gateway, consulte la guía de inicio rápido del RAK4631.
2. RAK4631 y RAK4631-R comparten el mismo hardware, aunque el RAK4631-R cuenta con firmware basado en RUI3, lo cual facilita el desarrollo de firmware optimizado usando las API de RUI3.
3. Para actualizar el RAK4631 a RUI3, siga las instrucciones indicadas en la documentación correspondiente.

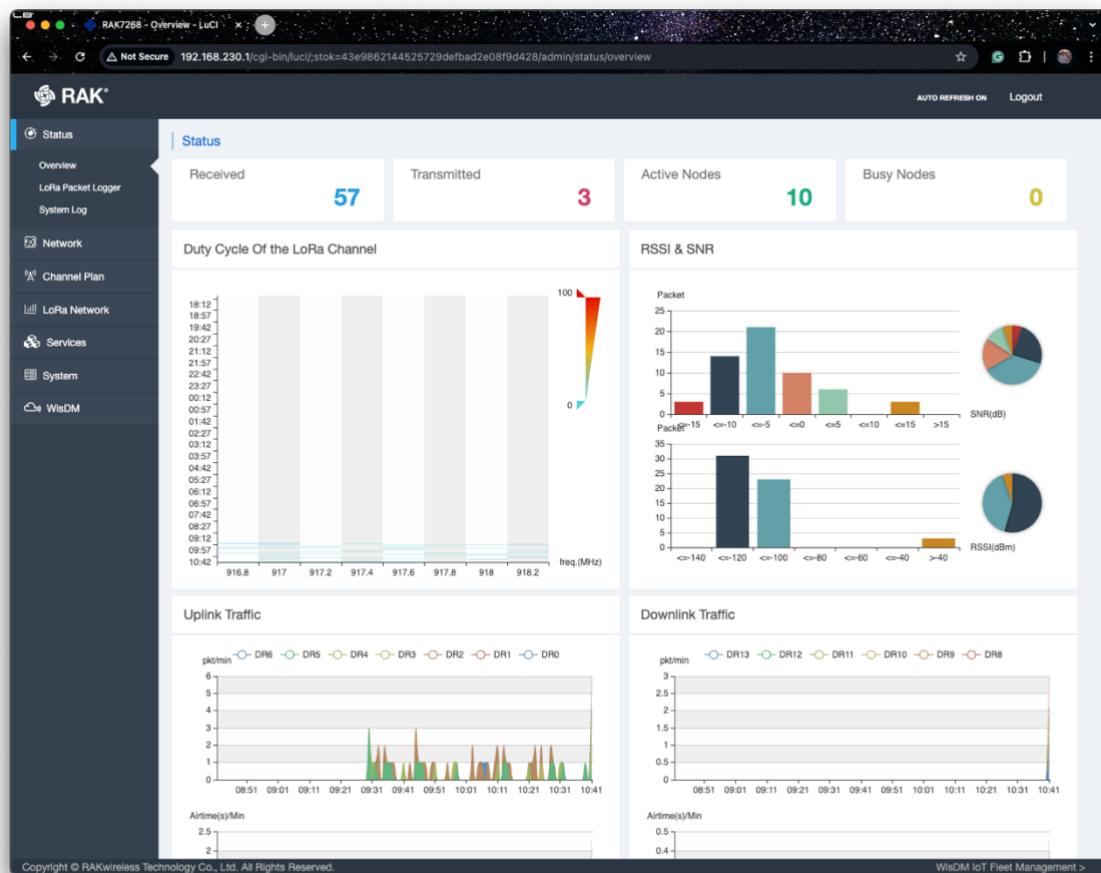


Figura 29 Dashboard de Gateway Rak. Fuente: Autores

7. Unión al Servidor AWS IoT LoRaWAN

1. Perfil del Dispositivo y Perfil del Servicio

- 2. Agregar Perfil del Dispositivo:** Para la configuración inicial del dispositivo, es necesario agregar un perfil que contenga las especificaciones necesarias para la comunicación mediante LoRaWAN.
- 3. Agregar Perfil del Servicio:** Este perfil permite definir los servicios que utilizará el dispositivo en la red.
- 4. Agregar Destino:** Antes de agregar el destino, configure la política IAM y el rol correspondiente para AWS IoT Core para LoRaWAN.

Para agregar un dispositivo al servicio AWS IoT, se requiere el DevEui, AppEui y AppKey, los cuales se pueden obtener a través de comandos AT, como se muestra a continuación:

```
AT+DEVEUI=0000000000000000
AT+APPEUI=0000000000000000
AT+APPKEY=00000000000000000000000000000000
```

5. Unirse al Servidor AWS IoT LoRaWAN

Para unir el dispositivo al servidor de AWS IoT LoRaWAN, use el siguiente comando:

```
AT+JOIN=1:0:10:8
```

Una vez conectado, es posible enviar mensajes de enlace ascendente utilizando el comando AT+SEND.

8. Verificación de la Operación - Ejemplo "Hello World"

Una vez completada la configuración, los dispositivos OTAA provisionados pueden unirse a la red y comenzar a enviar mensajes. Los mensajes provenientes de los dispositivos pueden ser recibidos por AWS IoT Core para LoRaWAN y luego reenviados al motor de reglas de IoT.

Para un ejemplo básico de aplicación "Hello World", siga las instrucciones de configuración de la función Lambda, que permite procesar los mensajes provenientes del dispositivo.

```
import base64
import json
import logging
import ctypes
import boto3

# Nombre de la función
FUNCTION_NAME = "RAK-HelloWorld"

# Tipos de datos en el payload
DATA_TYPES = 1
TYPE_TEMP = 0x67

# Configurar cliente iot-data para boto3
client = boto3.client('iot-data')
```

```
# Configurar logger
logger = logging.getLogger(FUNCTION_NAME)
logger.setLevel(logging.INFO)

def decode(event):
    data_base64 = event.get("PayloadData")
    data_decoded = base64.b64decode(data_base64)

    result = {
        "devEui": event.get("WirelessMetadata").get("LoRaWAN").get("DevEui"),
        "fPort": event.get("WirelessMetadata").get("LoRaWAN").get("FPort"),
        "freq": event.get("WirelessMetadata").get("LoRaWAN").get("Frequency"),
        "timestamp": event.get("WirelessMetadata").get("LoRaWAN").get("Timestamp")
    }

    if data_decoded[DATA_TYPES] == TYPE_TEMP:
        temp = (data_decoded[DATA_TYPES + 1] << 8) | (data_decoded[DATA_TYPES + 2])
        temp = ctypes.c_int16(temp).value
        result['temperature'] = temp / 10

    return result

def lambda_handler(event, context):
    data = decode(event)
    logger.info("Data: %s" % json.dumps(data))

    response = client.publish(
        topic = event.get("WirelessMetadata").get("LoRaWAN").get("DevEui") +
        "/project/sensor/decoded",
        qos = 0,
        payload = json.dumps(data)
    )
    return response
```

9. Creación de una Función Lambda

Para procesar los mensajes enviados por el dispositivo, se debe crear una función Lambda en la consola de AWS Lambda. La función se configurará con código en Python 3.8 o superior para decodificar la información de los sensores y publicar los resultados en un tópico MQTT de AWS IoT Core.

El siguiente código ejemplifica el manejo de mensajes del dispositivo, decodificando la información y publicando los datos procesados en un tópico MQTT:

Una vez que el código ha sido configurado, debe desplegarse y probarse utilizando la opción de pruebas en la consola de Lambda. Asegúrese de otorgar los permisos necesarios para que la función Lambda pueda publicar en el servicio AWS IoT.

10. Creación de la Regla de Destino

Finalmente, para que los datos del dispositivo puedan ser reenviados a su aplicación, se debe crear una regla IoT en la consola de AWS. Esta regla permite redirigir los mensajes a un tópico específico o a una función Lambda para su procesamiento posterior.

Procedimiento para Crear la Regla

1. Navegue a la consola de AWS IoT y seleccione **Actuar > Reglas**.
2. En la página de reglas, haga clic en **Crear**.
3. Configure una acción para **Reenviar un mensaje a un tópico AWS IoT** e ingrese el tópico `project/sensor/decoded`.
4. Agregue una acción adicional para invocar la función Lambda creada previamente.

De esta forma, la regla se asocia al destino creado, permitiendo que los datos sean procesados según las necesidades de la aplicación.

12.1. REPOSITORIO DE CÓDIGO

El código utilizado para el desarrollo de esta investigación se encuentra disponible en el siguiente repositorio de GitHub: https://github.com/Sharwin/iot_llm_react. A continuación, se describe el contenido de cada una de las carpetas del repositorio:

- **iot:** En esta carpeta se encuentra el código de Arduino para la programación del sensor IoT. Este código es el encargado de recolectar datos meteorológicos y enviarlos a la infraestructura de la nube para su procesamiento.
- **langsmith_export:** Contiene el código necesario para exportar los valores de LangSmith. LangSmith se utilizó como parte del proceso de análisis de los datos recolectados, permitiendo una exportación eficiente para su posterior evaluación.

- **randomforest:** Aquí se encuentra el código utilizado para entrenar el modelo basado en Random Forest. Este modelo fue empleado para generar predicciones meteorológicas a partir de los datos recolectados por los sensores IoT.
- **react:** Esta carpeta contiene el código del agente ReAct, implementado para optimizar la toma de decisiones en el sistema. El agente ReAct permite inferir y planificar acciones basadas en los datos meteorológicos, integrando capacidades de razonamiento y actuación dentro del sistema IoT.

Este repositorio facilita el acceso a todos los componentes utilizados en la investigación, permitiendo a otros investigadores y desarrolladores replicar y mejorar los resultados obtenidos.